그레이엄 방어적 투자자 기준의 한국 주식시장 적용

종목 선별과 백테스트 (2017–2026)

분석 기준일 2026-10-16 · 작성일 2026-10-18 · Python(pandas) · MySQL

요약

분석 범위 현재 상장 보통주 2,532 종목과 2015 년 이후 상장폐지된 424 종목을 합친 2,956 종목을 대상으로 했고, 이 중 연간 재무(2013~2025 회계연도)를 확보한 2,320 종목을 분석했다. 주가는 2015-01-02 부터 2026-10-16 까지 사용했다.

방법 그레이엄 방어적 투자자의 7 가지 기준(규모, 유동비율, 장기부채, 이익 안정성, 이익 성장, PER, PBR)을 종목별로 적용했다. 현재 시점 스크리닝, 매년 4 월 1 일에 통과 종목을 사서 1 년 보유하는 정기 방식(A), 재무 조건을 통과한 종목이 매수 상한가 아래로 내려온 날 사는 신호 방식(B)을 비교했고, 벤치마크는 같은 조건의 전종목 동일가중이다.

두 가지 버전 실제 버전은 10 년 연속 흑자와 10 년 성장을 요구한다. 변형 버전은 일정 수 이상의 종목을 고르고 더 긴 기간을 검증하기 위해 이 요구 기간을 4 년으로 줄였다.

현재 시점 결과 2026-10-16 기준 모든 조건을 통과한 종목은 실제 버전 67 개, 변형 버전 106 개다.

백테스트 결과 변형 버전(2017-04~2026-10, 9.5 년)의 연평균 수익률은 정기 3.1%, 신호 3.1%, 벤치마크 6.1% 였다. 실제 버전(2023-04~, 3.5 년, 리밸런싱 4 회)은 정기 5.6%, 신호 5.6%, 벤치마크 3.8%로 앞섰지만, 기간이 짧고 초과수익이 2025~2026 년에만 나타났다.

결론 이 표본에서 그레이엄 기준만으로는 벤치마크를 이기지 못했다. 기준값을 바꾼 6 가지 조합 모두 벤치마크보다 낮았고, 2025 년 이전만 보면 격차가 더 컸다. 원인과 한계는 5 장과 6 장에 정리했다.

목차

1. 연구 개요
2. 데이터
3. 분석 방법
4. 결과
5. 결과 해석: 왜 이런 결과가 나왔는가
6. 한계
7. 결론
8. 부록: 코드 전체

1. 연구 개요

벤저민 그레이엄이 『현명한 투자자』에서 제시한 방어적 투자자 기준은 가치투자의 출발점으로 널리 알려져 있다. 이 연구는 그 기준을 한국 상장 종목에 그대로 적용했을 때 어떤 종목이 걸리는지, 같은 규칙을 과거 시점마다 적용했다면 성과가 어땠는지를 데이터로 확인하는 것을 목적으로 한다.

세 가지 질문에 답하도록 구성했다.

1. 지금 시점에 그레이엄의 7 가지 기준을 모두 통과하는 종목은 몇 개이고, 어떤 종목인가.
2. 재무 조건은 통과했지만 가격이 비싼 종목은 시가총액이 얼마까지 내려오면 사도 되는가(매수 상한가).

3. 과거 시점마다 같은 규칙으로 종목을 골랐다면 전종목 동일가중 대비 성과는 어땠는가. 결과는 얼마나 믿을 수 있는가.

같은 데이터와 같은 구조로 버핏식 기준을 적용한 별도 보고서를 함께 작성했다.

2. 데이터

2.1 구성과 출처

모든 데이터는 로컬 MySQL(stock_db)에 저장하고 Python 으로 읽어 분석했다. 수집 과정에서 사이트 접속이 차단되지 않도록 요청 간격 조절, 날짜별 캐시, 재시도, 중단 후 이어받기를 넣었다.

표 1. 데이터 구성

구분	내용	출처	기간·규모
종목 목록·시가총액	현재 상장 보통주의 종가, 시가총액, 시장구분, 업종	KRX(pykrx), WICS	2026-10-16 기준 2,532 종목
수정주가	일별 종가(액면분할 등 반영)	네이버 금융 차트	2015-01-02 ~ 2026-10-16
장기 재무	연간 재무제표(연결 우선, 별도 보완), 단위 억원	OpenDART	2013~2025 회계연도, 12 월 결산 법인, 2,320 종목
보조 재무	지배주주 지분 등 연간 재무	FnGuide	최근 3 개 회계연도(지배주주 기준 점검용)
상장폐지 종목	2015 년 이후 폐지 종목 목록, 일별 종가·시가총액	FinanceDataReader, KRX	424 종목, 시가총액 단위 원 확인

2.2 재무 데이터 처리

OpenDART 의 사업보고서 한 건에는 당기·전기·전전기 3 개 연도가 들어 있다. 그래서 2025, 2022, 2019, 2016, 2015 사업연도를 요청해 2013 회계연도까지 채웠다. 연결재무제표를 먼저 받고 없으면 별도재무제표로 대체했다. 결산월이 12 월이 아닌 32 종목은 연도 기준이 달라서 제외했다.

계정 이름은 회사마다 다르게 표기될 수 있어서, 표준 계정 ID 를 먼저 찾고 없으면 계정명으로 찾도록 매핑했다. 필요한 계정이 얼마나 확보됐는지는 아래 표로 확인했다.

표 2. 분석에 쓴 계정의 확보 종목 수

계정	의미	확보 종목 수
sales	매출액	2,278
ni	당기순이익	2,316
equity	자본총계	2,320
ca	유동자산	2,258
cl	유동부채	2,258
ltd	비유동부채(장기부채 대용)	2,258

종목별로 확보한 연간 재무의 연수는 13 년 1,465 종목, 10 년 이상 1,677 종목, 5 년 이상 2,108 종목이다. 실제 버전은 10 년 이상, 변형 버전은 4 년 이상의 재무가 있어야 평가할 수 있다.

2.3 생존편향 완화

현재 상장 종목만 쓰면 백테스트 기간에 상장폐지된 종목이 빠져 수익률이 실제보다 높게 나온다. 이를 줄이려고 FinanceDataReader 의 상장폐지 목록에서 주권(보통주 등)만 골라 2015 년 이후 폐지된 424 종목의 가격과 시가총액을 넣었다. KRX 가 가격 요청을 로그인 없이는 받지 않아서 KRX 로그인 세션 방식으로 수집했다.
시가총액을 종가로 나눈 값이 상장주식수와 일치하는 것을 확인해 단위가 원임을 검증했다.

2.4 가격 데이터 확인

네이버 금융 수정주가가 액면분할을 반영하는지 삼성전자로 확인했다. 2015-01-02 종가가 26,600 원, 2018-04-27 종가가 53,000 원으로, 2018 년 5 월의 50 대 1 액면분할이 반영된 값이다.

3. 분석 방법

3.1 그레이엄 방어적 투자자 기준

아래 표는 7 가지 기준과 이 연구에서의 구현이다. 숫자로 정해지지 않은 부분과 원화 환산이 필요한 부분은 분석자가 값을 정했고, 모든 값은 코드 상단 변수로 두어 바꿔 볼 수 있다.

표 3. 기준과 구현

기준	취지	구현
1. 적정 규모	너무 작은 회사 제외	최근 연간 매출 1,000 억원 이상(원전의 미화 1 억 달러를 환산한 값, 분석자 설정)
2. 유동비율	단기 재무 건전성	유동자산 / 유동부채 ≥ 2.0
3. 장기부채	장기부채가 순유동자산 이내	비유동부채 ≤ 유동자산 − 유동부채
4. 이익 안정성	장기간 적자 없음	실제 버전 10 년, 변형 버전 4 년 연속 당기순이익 흑자
5. 이익 성장	장기 이익 증가	당기순이익이 1/3 이상 증가(주당이익 자료가 없어 순이익 사용)
6. PER	이익 대비 낮은 가격	최근 3 년 평균 순이익 기준 PER ≤ 15
7. PBR	장부가 대비 낮은 가격	PER × PBR ≤ 22.5 (PBR 1.5 × PER 15)

원전의 배당 20 년 연속 조건은 과거 배당 데이터가 DB 에 없어 적용하지 않았다.

3.2 실제 버전과 변형 버전

실제 버전은 원전에 가깝게 10 년 연속 흑자와 10 년 성장을 요구한다. 그런데 DART 재무가 2013 회계연도부터라 평가가 가능한 시점이 2023 년 4 월 이후로 한정되고, 리밸런싱이 4 번뿐이다. 일정 수 이상의 종목을 안정적으로 고르고 더 긴 기간을 검증하기 위해, 흑자와 성장을 요구하는 기간을 4 년으로 줄인 변형 버전을 함께 만들었다. 두 버전은 이 두 항목과 이익 성장 비교 방식만 다르고, 나머지 기준은 같다.

표 4. 두 버전의 차이

항목	실제 버전	변형 버전
연속 흑자 요구	최근 10 년	최근 4 년
이익 성장 비교	최근 10 년 구간에서 처음 3 년 평균 대비 마지막 3 년 평균	최근 최대 10 년 구간에서 첫해 대비 마지막 해 (최소 4 년)
PER 평균 기간	3 년	3 년
백테스트 시작	2023-04-03	2017-04-03

항목	실제 버전	변형 버전
리밸런싱 횟수	4 회	10 회
2026-10-16 통과 종목	67 개	106 개

3.3 시점 정합성

- 결산일에서 90 일이 지난 뒤에야 그 재무를 알 수 있다고 가정했다. 미래 정보를 쓰지 않기 위해서다. 12 월 결산 사업보고서는 3 월 말까지 나오므로 리밸런싱을 매년 4 월 1 일 이후 첫 거래일로 정했다.
- 과거 시가총액은 현재 주식수 × 그날의 수정주가로 근사했다. 상장폐지 종목은 수집한 시가총액(MarCap)을 썼다.
- 상장폐지된 종목은 폐지 이후 가격이 없어 그날 이후 후보에서 자동으로 빠진다.

3.4 매수 상한가

PER ≤ 15 와 PER × PBR ≤ 22.5 를 시가총액 기준으로 풀면 다음과 같다. 주식수가 약분되어 사라지므로 주식수 추정 오차의 영향을 받지 않는다.

> 시가총액 ⩽ min(15 × 3 년 평균 순이익 , $\sqrt{}$(22.5 × 3 년 평균 순이익 × 자본총계))

재무 조건을 통과한 종목의 시가총액이 이 값 아래로 내려오는 날이 진입 시점이다. 현재 시점에서는 시가총액이 이 값보다 몇 % 높은지를 gap 으로 계산했다.

3.5 백테스트 설계

- **정기 방식(A)**: 매년 4 월 1 일 이후 첫 거래일에 스크리닝해 통과 종목을 동일가중으로 사고, 다음 리밸런싱까지 보유한다. 통과 종목이 없으면 현금으로 둔다.
- **신호 방식(B)**: 같은 날 재무 조건만 통과한 종목을 후보로 두고 종목마다 같은 금액을 배정한다. 시가총액이 매수 상한가 아래로 처음 내려온 날 사고, 매수가 대비 +50%에 닿으면 팔며, 아니면 다음 리밸런싱 날 정리한다. 매수 전 배정금은 현금(수익률 0)으로 둔다. +50% 청산 기준은 분석자가 정한 값이다.
- **벤치마크**: 같은 날짜에 같은 방식으로 굴린 전종목 동일가중(코스피 지수가 아니다).
- **거래비용**: 전량 교체 시 0.3%(왕복)를 가정했다. 슬리피지, 거래정지, 호가 단위는 반영하지 않았다.
- **성과 지표**: 누적수익률, 연평균 수익률(CAGR), 최대낙폭(MDD), 연 변동성(일수익률 표준편차 × $\sqrt{}$252), 샤프 비율(무위험수익률 0, 일수익률 평균 / 표준편차 × $\sqrt{}$252).

3.6 결과를 점검하는 방법

- 연도별 수익률: 초과수익이 한 해에 몰려 있는지 본다.
- 구간별 성과: 2025 년 이전과 이후로 나눠 같은 결론인지 본다.
- 민감도: PER × PBR 상한(18, 22.5, 30)과 유동비율 하한(1.5, 2.0)을 바꿔 결과가 유지되는지 본다.
- 지배주주 기준 점검: 연결 자본에는 비지배지분이 포함돼 PBR 이 낮게 나올 수 있어, FnGuide 의 지배주주 지분으로 PBR 을 다시 계산해 통과 여부가 바뀌는지 본다.
- 한계 자동 점검: 기간, 리밸런싱 횟수, 매수 종목 수, 진입 건수 등이 부족하면 코드가 경고를 출력한다.

4. 결과

4.1 현재 시점 스크리닝 (기준일 2026-10-16)

재무를 확보한 현재 상장 종목 2,319 개를 대상으로 7 가지 기준을 적용했다. 기준별 통과 종목 수는 아래와 같다.

표 5. 기준별 통과 종목 수

기준	실제 버전	변형 버전
적정 규모(매출 1,000 억원 이상)	1,342	1,342
유동비율 2.0 이상	846	846
장기부채 ≤ 순유동자산	1,269	1,269
이익 안정성(흑자 연속)	421	843
이익 성장(1/3 이상)	453	602
PER(3 년 평균) ≤ 15	778	778
PER × PBR ≤ 22.5	1,688	1,688
재무 조건 5 가지 통과	100	163
7 가지 모두 통과	**67**	**106**

변형 버전의 106 종목 가운데 100 종목은 현재 배당을 지급하고 있다. 7 가지 가운데 가장 많은 종목을 걸러내는 기준은 이익 안정성과 이익 성장, 유동비율이다.

지배주주 기준 점검

DART 의 자본총계와 순이익은 비지배지분을 포함한 연결 값이고 시가총액은 지배주주 지분만 반영한다. 지주회사처럼 비지배지분이 큰 회사는 PBR 이 실제보다 낮게 계산된다. 그래서 FnGuide 의 지배주주 지분으로 PBR 을 다시 계산했다. DART 와 FnGuide 의 자본총계, 순이익은 겹치는 5,000 여 건의 약 99.6%에서 3% 이내로 일치해 단위와 기준은 같았다. 변형 버전 통과 106 종목 가운데 지배주주 기준으로도 통과하는 종목은 93 개다. 이 계산은 PBR 과 PER × PBR 만 바꾸고 PER 은 연결 순이익을 그대로 썼다.

표 6. 지배주주 기준으로 PBR 이 크게 달라진 종목

종목	PBR(연결)	PBR(지배주주)	PER×PBR(연결)	PER×PBR(지배주주)
아이디스홀딩스	0.16	0.39	0.37	0.88
오리온홀딩스	0.27	0.56	0.99	2.07
매일홀딩스	0.16	0.32	0.32	0.64
영원무역홀딩스	0.43	0.75	1.60	2.80
SNT 홀딩스	0.30	0.53	1.06	1.83
KPX 홀딩스	0.24	0.35	0.73	1.09
아이디스	0.49	0.72	4.06	5.99
유비쿼스홀딩스	0.37	0.53	1.86	2.69
미창석유	0.44	0.58	1.67	2.20
제이엠티	0.23	0.29	0.59	0.75

지배주주 기준으로 바꿔도 이 10 종목은 모두 PER × PBR 22.5 이하를 유지했다. 변형 버전 통과 종목 중 FnGuide 에 지배주주 자본이 없는 종목(SHD, 세원물산 등)은 비교하지 못해 탈락으로 계산했다.

가격 대기 종목(관심 목록)

재무 조건 5 가지는 통과했지만 시가총액이 매수 상한가보다 높은 종목이다. gap 이 0 에 가까울수록 가격이 조금만 내려와도 진입 신호가 나온다.

표 7. 관심 목록 상위 8 종목 (변형 버전)

종목	시장	3 년 평균 PER	PBR	PER×PBR	gap
아가방컴퍼니	KOSDAQ	15.2	0.95	14.5	+1.5%
와이지엔터테인먼트	KOSDAQ	15.7	1.21	19.0	+4.6%
JYP Ent.	KOSDAQ	11.4	2.22	25.4	+6.2%
감성코퍼레이션	KOSDAQ	10.7	2.37	25.5	+6.5%
삼화전기	KOSPI	16.4	1.71	28.0	+11.6%
네오팜	KOSDAQ	14.9	1.89	28.2	+11.9%
싸이맥스	KOSDAQ	17.1	1.24	21.2	+14.2%
NICE 평가정보	KOSPI	14.1	2.10	29.7	+14.9%

4.2 백테스트 구성

변형 버전은 2017-04-03 부터 2026-10-16 까지 9.5 년, 리밸런싱 10 회다. 정기 방식은 평균 78.7 종목을 샀고 신호 방식은 943 건을 진입했다. 실제 버전은 2023-04-03 부터 3.5 년, 리밸런싱 4 회이며 정기 방식 평균 68.0 종목, 신호 방식 진입 306 건이다.

표 8. 리밸런싱별 종목 수 (변형 버전)

리밸런싱일	후보 종목	정기 방식 매수	신호 방식 진입
2017-04-03	87	39	45
2018-04-02	99	48	74
2019-04-01	116	71	91
2020-04-01	95	72	72
2021-04-01	117	51	63
2022-04-01	163	91	122
2023-04-03	142	103	117
2024-04-01	147	101	114
2025-04-01	155	110	125
2026-04-01	163	101	120

표 9. 리밸런싱별 종목 수 (실제 버전)

리밸런싱일	후보 종목	정기 방식 매수	신호 방식 진입
2023-04-03	83	60	67
2024-04-01	90	68	75
2025-04-01	100	78	85
2026-04-01	100	66	79

4.3 성과 비교

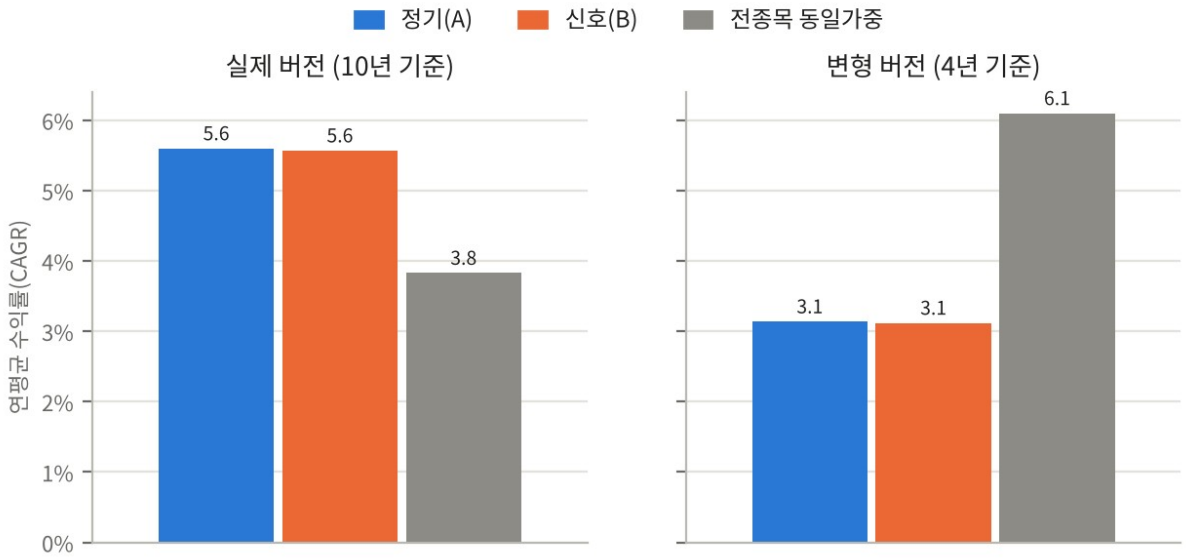


그림 1. 연평균 수익률(CAGR) 비교

표 10. 성과 — 변형 버전 (2017-04 ~ 2026-10)

지표	정기(A)	신호(B)	전종목 동일가중
누적수익률	34.3%	34.0%	75.8%
연평균 수익률(CAGR)	3.1%	3.1%	6.1%
최대낙폭(MDD)	-47.6%	-25.3%	-48.2%
연 변동성	18.8%	10.4%	21.3%
샤프 비율(무위험 0)	0.27	0.36	0.39
평가 기간	9.5 년	9.5 년	9.5 년

표 11. 성과 — 실제 버전 (2023-04 ~ 2026-10)

지표	정기(A)	신호(B)	전종목 동일가중
누적수익률	21.2%	21.1%	14.2%
연평균 수익률(CAGR)	5.6%	5.6%	3.8%
최대낙폭(MDD)	-22.0%	-14.8%	-35.0%
연 변동성	19.2%	11.5%	22.9%
샤프 비율(무위험 0)	0.40	0.55	0.29
평가 기간	3.5 년	3.5 년	3.5 년

변형 버전에서 정기(A)와 신호(B)의 연평균 수익률은 모두 3.1%로 벤치마크 6.1%의 절반 수준이다. 샤프 비율도 0.27, 0.36 으로 벤치마크 0.39 보다 낮다. 정기(A)의 최대낙폭은 -47.6%로 벤치마크(-48.2%)와 거의 같아 위험도 줄이지 못했다. 신호(B)는 변동성(10.4%)과 최대낙폭(-25.3%)이 낮은데, 매수 전 배정금을 현금으로 두는 구조의 영향으로 보인다(추정).

실제 버전은 정기·신호 모두 벤치마크보다 높고 낙폭도 작다. 그러나 기간이 3.5 년이고 리밸런싱이 4 번뿐이며, 아래에서 보듯 초과수익이 2025~2026 년에 집중돼 있어 결론을 내기에는 표본이 작다.

4.4 연도별 수익률

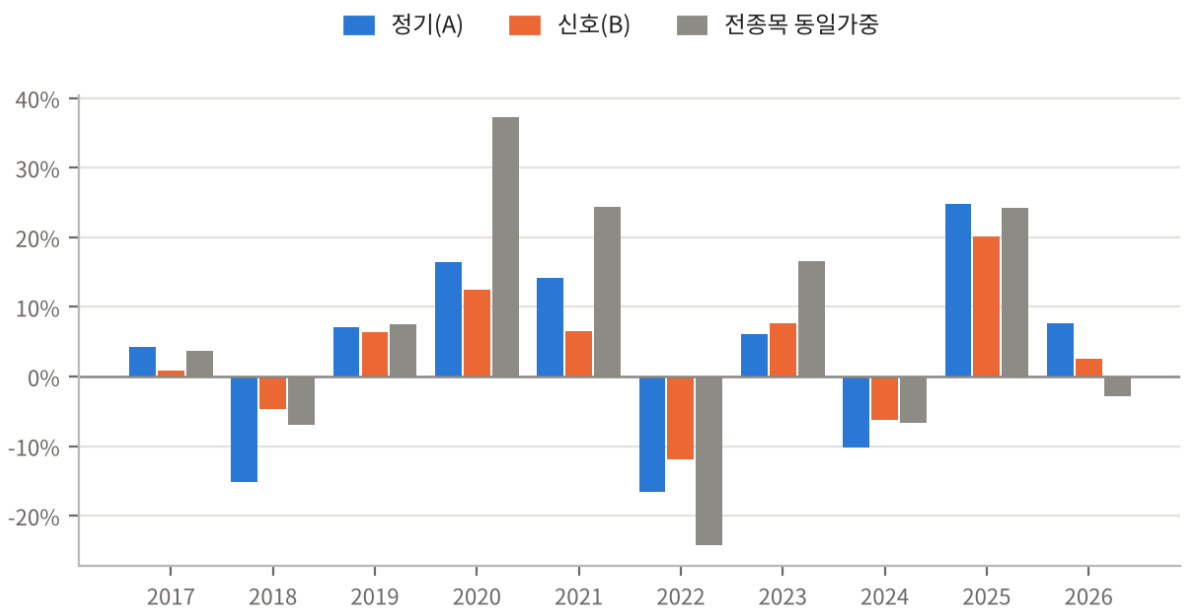


그림 2. 연도별 수익률 — 변형 버전

표 12. 연도별 수익률 — 변형 버전

연도	정기(A)	신호(B)	전종목 동일가중	A – 벤치
2017(4 월 이후)	4.3%	0.8%	3.6%	+0.7%p
2018	-15.2%	-4.7%	-7.0%	-8.2%p
2019	7.2%	6.4%	7.5%	-0.3%p
2020	16.5%	12.5%	37.3%	-20.8%p
2021	14.2%	6.5%	24.4%	-10.2%p
2022	-16.6%	-11.9%	-24.2%	+7.6%p
2023	6.1%	7.6%	16.6%	-10.4%p
2024	-10.2%	-6.2%	-6.6%	-3.7%p
2025	24.7%	20.1%	24.2%	+0.5%p
2026(10/16 까지)	7.7%	2.6%	-2.9%	+10.6%p

2017 년은 4 월 리밸런싱 이후, 2026 년은 10 월 16 일까지의 수익률이다.

정기(A)가 벤치마크보다 나은 해는 10 년 중 4 년(2017, 2022, 2025, 2026)이다. 2020 년(-20.8%p), 2021 년(-10.2%p), 2023 년(-10.4%p)에 크게 뒤졌고, 시장이 하락한 2022 년(+7.6%p)과 부진했던 2026 년(+10.6%p)에는 앞섰다.

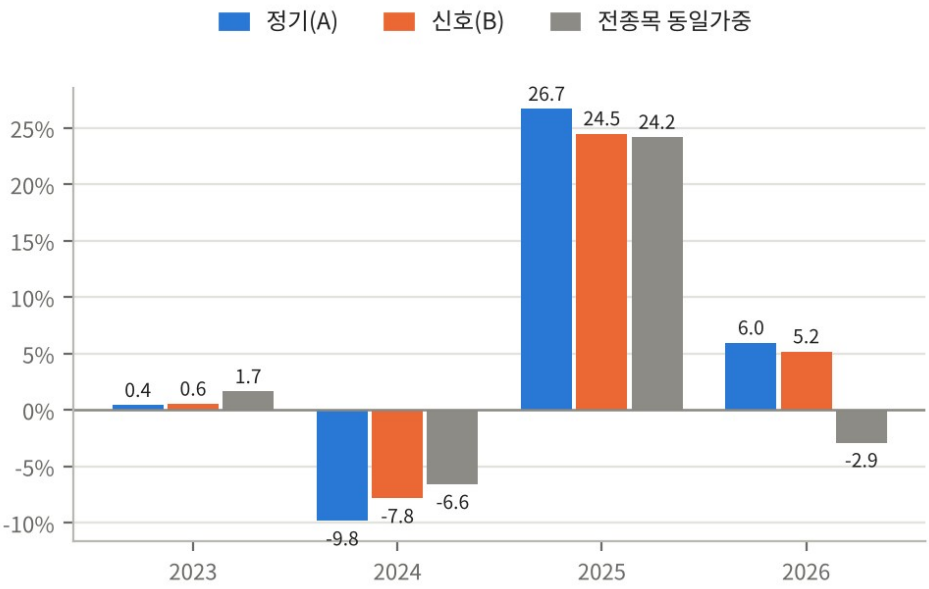


그림 3. 연도별 수익률 — 실제 버전

표 13. 연도별 수익률 — 실제 버전

연도	정기(A)	신호(B)	전종목 동일가중	A – 벤치
2023(4 월 이후)	0.4%	0.6%	1.7%	-1.2%p
2024	-9.8%	-7.8%	-6.6%	-3.2%p
2025	26.7%	24.5%	24.2%	+2.5%p
2026(10/16 까지)	6.0%	5.2%	-2.9%	+8.8%p

2023 년은 4 월 리밸런싱 이후, 2026 년은 10 월 16 일까지의 수익률이다.

4.5 구간별 성과: 2025 년 이전과 이후

2025 년 이후 시장이 크게 올랐기 때문에, 그 구간을 빼도 같은 결론인지 확인했다. 연도별 수익률을 이어 붙여 구간 수익률을 계산했다.

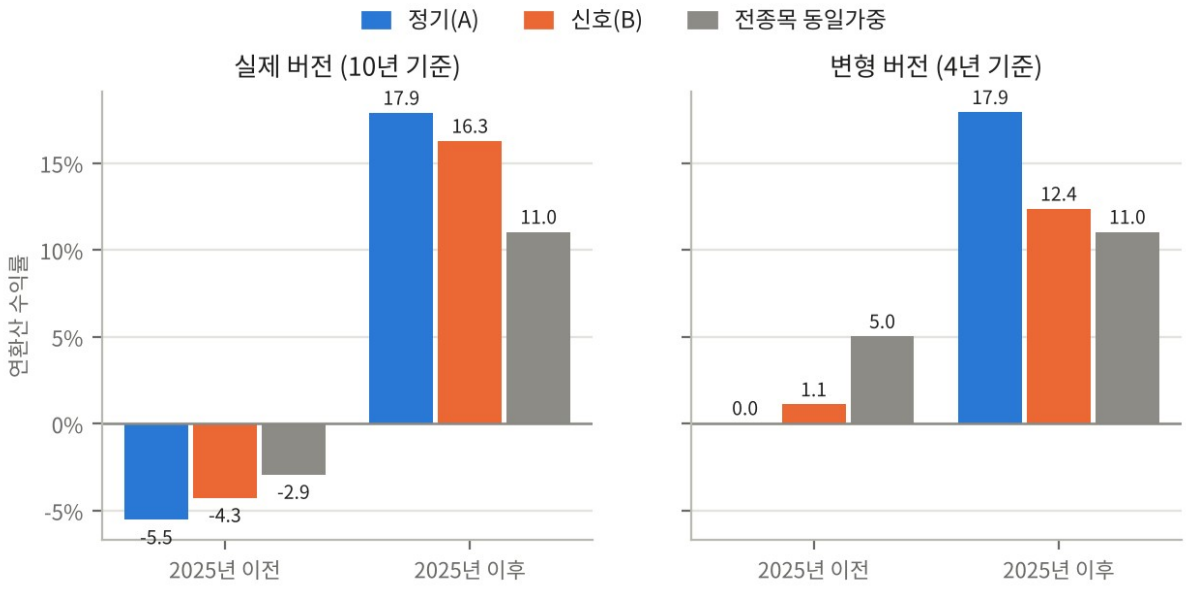


그림 4. 구간별 연환산 수익률

표 14. 구간별 누적수익률 (연도별 수익률 복리 환산)

버전	구간	정기(A)	신호(B)	전종목 동일가중
변형	2025 년 이전 (2017-04 ~ 2024-12)	+0.3%	+8.9%	+46.2%
변형	2025 년 이후 (2025-01 ~ 2026-10)	+34.3%	+23.2%	+20.6%
실제	2025 년 이전 (2023-04 ~ 2024-12)	-9.4%	-7.3%	-5.0%
실제	2025 년 이후 (2025-01 ~ 2026-10)	+34.3%	+31.0%	+20.6%

2025 년을 빼면 오히려 격차가 벌어진다. 변형 버전은 2025 년 이전 7.7 년 동안 정기(A)가 누적 +0.3%에 그친 반면 벤치마크는 +46.2%였다. 정기(A)가 벤치마크를 앞선 구간은 2025 년 이후뿐이다. 실제 버전의 우위도 같은 구간에서만 나타나, 우연일 가능성을 배제하기 어렵다.

4.6 민감도

PER × PBR 상한(18, 22.5, 30)과 유동비율 하한(1.5, 2.0)을 바꿔 변형 버전을 다시 돌렸다.

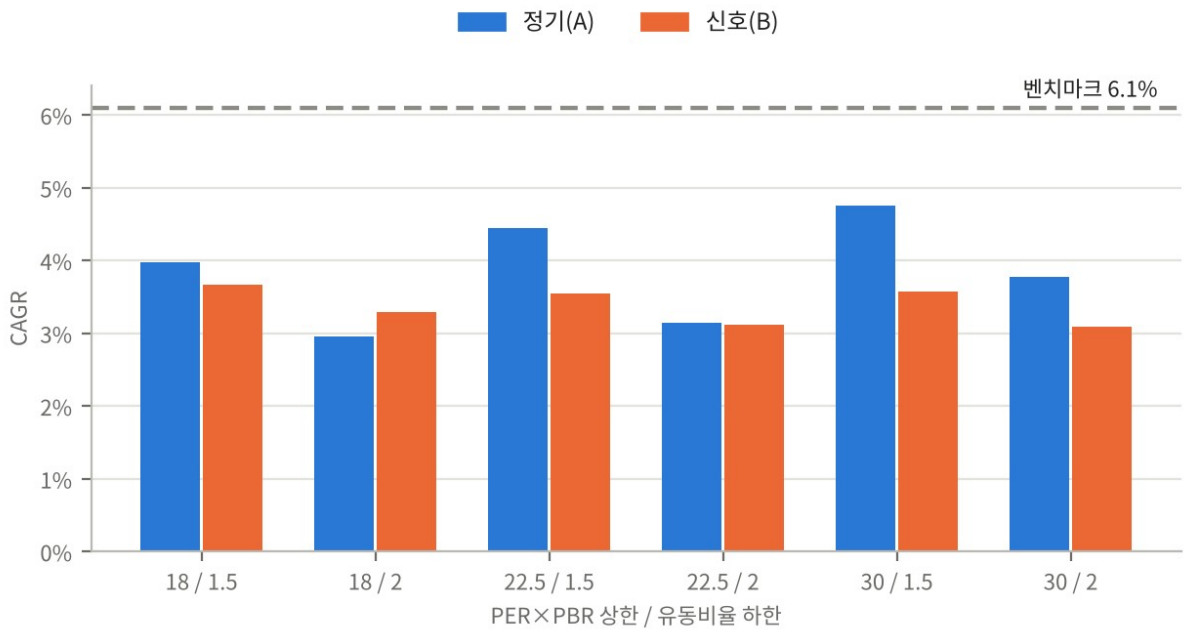


그림 5. 기준값에 따른 연평균 수익률 (변형 버전)

표 15. 민감도 (변형 버전)

PER×PBR 상한	유동비율 하한	CAGR 정기(A)	CAGR 신호(B)	CAGR 벤치	평균 매수 종목
18	1.5	4.0%	3.7%	6.1%	108.4
18	2.0	3.0%	3.3%	6.1%	75.8
22.5	1.5	4.5%	3.6%	6.1%	113.1
22.5	2.0	3.1%	3.1%	6.1%	78.7
30	1.5	4.8%	3.6%	6.1%	117.3
30	2.0	3.8%	3.1%	6.1%	81.9

여섯 가지 조합 모두 정기(A)가 3.0~4.8%, 신호(B)가 3.1~3.7%로 벤치마크 6.1%에 못 미쳤다. 기준을 느슨하게 할수록 정기(A)가 조금 나아졌지만 벤치마크를 넘지는 못했다. 기준값을 조금 바꿔도 결론이 같다는 점에서 이 결과는 특정 기준값에서만 나타난 우연이라기보다 일관된 경향이다.

5. 결과 해석: 왜 이런 결과가 나왔는가

아래는 결과를 설명할 수 있는 원인 후보다. 데이터로 직접 확인한 것과 가설인 것을 구분해 적었다.

5.1 확인된 사실

- 변형 버전에서 정기(A)와 신호(B)는 연평균 3.1%로 벤치마크 6.1%보다 낮았다. 기준값을 바꾼 6 가지 조합 모두 같았다.
- 정기(A)가 벤치마크보다 크게 뒤진 해는 2020 년, 2021 년, 2023 년이다. 시장이 하락한 2022 년과 2026 년에는 앞섰다.
- 2025 년 이전만 보면 정기(A)는 7.7 년 동안 거의 수익이 없었다(누적 +0.3%). 앞선 구간은 2025 년 이후뿐이다.
- 신호(B)는 변동성과 최대낙폭이 작지만 수익률은 벤치마크에 못 미쳤다.

5.2 원인 후보

(1) 상승장에서 뒤처지는 구조

낮은 가격과 재무 안정성을 요구하는 기준은 시장이 급반등하거나 성장 종목이 주도할 때 그 종목들을 거의 담지 못한다. 2020 년과 2021 년의 격차가 이를 보여준다. 반대로 시장이 내려가는 구간에서는 방어력을 보였다. 어떤 종목군이 상승을 주도했는지는 이 DB 에 업종별 분석을 하지 않아 확인하지 못했다.

(2) 저평가 종목의 구성

PER × PBR 기준을 통과한 종목에 지주회사가 많이 포함됐다(표 6). 지주회사는 자회사 가치에 비해 시가총액이 낮게 형성되는 경향이 있어, 숫자상 저평가가 쉽게 해소되지 않을 수 있다. 이는 이 연구에서 검증하지 않은 설명이다. 확인된 것은 연결 기준 PBR 이 지배주주 기준 PBR 보다 낮게 나오는 종목이 있었다는 점이다(예: 아이디스홀딩스 0.16 대 0.39). 다만 백테스트는 연결 기준으로 돌렸기 때문에, 지배주주 기준으로 다시 돌리면 선정 종목과 성과가 바뀔 수 있다.

(3) 벤치마크의 성격

벤치마크가 코스피 지수가 아니라 DB 의 전종목 동일가중이다. 동일가중은 소형주 비중이 커서 소형주가 강한 구간에서 유리하다. 코스피 지수는 DB 에 없어서 지수와의 비교는 하지 않았다.

(4) 이익 지표의 한계

주당이익 대신 당기순이익을 썼고, 일회성 손익이 포함된다. 3 년 평균으로 완화했지만 업황 변동이 큰 업종에서는 PER 이 왜곡될 수 있다. 이익 성장 기준은 처음 값이 작으면 증가율이 비정상적으로 커져(최대 2,000 배 이상) 성장성 지표로 읽기 어려운 종목이 있었다. 통과 여부(1/3 이상)에는 영향이 없다.

(5) 시가총액 근사

과거 시가총액을 현재 주식수로 근사했다. 유상증자나 자사주 소각이 많았던 종목은 과거 시점의 PER, PBR 과 매수 상한가가 틀어질 수 있다.

(6) 신호 방식의 현금 보유

신호 방식은 매수 신호가 오기 전까지 배정금을 현금으로 둔다. 낙폭과 변동성이 작은 것은 이 때문으로 보이며(추정), 그만큼 시장 상승분도 놓친다.

5.3 그레이엄과 버핏은 왜 유명한 투자자가 되었는가

이 결과를 보면 그레이엄이나 버핏의 방법이 효과가 없다는 뜻인지 의문이 생긴다. 이 연구는 그 둘의 투자를 재현한 것이 아니라서 그렇게 결론 낼 수 없다. 이유는 다음과 같다.

- **기계적 스크리닝과 실제 투자의 차이**: 이 연구는 그레이엄 기준 중 숫자로 옮길 수 있는 부분을 분석자가 수치로 정해 기계적으로 적용했다. 실제 투자는 기업별 청산가치 분석, 안전마진 판단, 분산 등 판단이 함께 들어간다.
- **시대의 차이**: 그레이엄이 활동한 20 세기 중반에는 재무 정보를 구하기 어려워 숫자만으로 싼 종목을 찾는 것 자체가 경쟁력이었다. 지금은 누구나 같은 스크리닝을 하기 때문에 같은 기준만으로 초과수익을 얻기 어렵다는 지적이 널리 알려져 있다. 이 연구에서 검증한 것은 아니다.
- **버핏의 방식**: 버핏은 숫자 스크린이 아니라 사업의 경쟁력과 경영진을 판단해 소수 종목에 집중하고 오래 보유했다. 보험사 자금을 굴리고 기업 전체를 인수하는 등 개인이 숫자만 보고 따라 하기 어려운 구조도 있다. 그의 장기 성과는 높은 것으로 알려져 있으나 이 대화와 연구에서 수치를 확인하지는 않았다.
- **시장의 차이**: 한국 시장은 지배구조 문제로 저 PBR 이 오래 지속되는 종목이 많다는 지적이 있다. 이 경우 숫자상 싼 종목이 싼 채로 남을 수 있다. 이 연구에서 검증하지 않은 가설이다.

정리하면, 이 결과는 "그레이엄 기준을 기계적으로 한국 시장에 적용하면 이 표본에서는 시장을 이기지 못했다"는 의미로 읽어야 한다.

6. 한계

- **생존편향이 일부 남는다**: 상장폐지 종목을 넣었지만 재무를 확보하지 못한 폐지 종목(감사의견 거절, 합병 등)은 분석에서 빠졌다. 폐지 종목의 종가가 수정주가인지는 확인하지 못했다. 현재 상장 종목의 수정주가는 삼성전자 사례로 확인했다.
- **시가총액은 근사값**: 현재 주식수 × 과거 수정주가다.
- **이익 지표**: 주당이익이 아니라 연결 당기순이익(비지배지분 포함)이다. 지배주주 기준 점검은 현재 시점 스크리닝에서만 했고 백테스트에는 반영하지 않았다.
- **12 월 결산 법인만 분석**: 결산월이 다른 32 종목은 제외했다.
- **배당 조건 미적용**: 20 년 연속 배당은 과거 배당 데이터가 없어 뺐다.
- **기준값은 분석자 설정**: 규모 1,000 억원, 신호 방식의 +50% 청산 등은 분석자가 정한 값이다. 민감도에서 일부(PER×PBR 상한, 유동비율)만 바꿔 봤다.
- **거래 제약 미반영**: 거래비용은 0.3% 가정이고 슬리피지, 거래정지, 호가 단위는 반영하지 않았다.
- **벤치마크**: 전종목 동일가중이며 코스피 지수와의 비교는 하지 않았다.
- **표본 크기**: 실제 버전은 3.5 년, 리밸런싱 4 회다. 변형 버전도 9.5 년, 10 회라서 시장 국면을 충분히 다 보지 못했다. 위의 수치는 통계적 검정이 아니다.

7. 결론

- 그레이엄 방어적 투자자 기준은 한국 시장 2017~2026 년 표본에서 전종목 동일가중 벤치마크를 일관되게 이기지 못했다(변형 버전 연 3.1% 대 6.1%, 기준값 6 가지 조합 모두 동일).
- 실제 버전의 우위(연 5.6% 대 3.8%)는 2025~2026 년에 집중돼 있고 기간이 짧아서 전략의 효과로 보기 어렵다.
- 신호 방식은 낙폭과 변동성이 작아 방어적 성격은 확인됐지만, 그만큼 수익률을 희생했다.
- 이 기준을 단독 매매 전략으로 쓰기보다 재무 건전성과 가격 부담을 거르는 스크리닝으로 쓰고, 다른 요인과 결합하는 연구의 출발점으로 삼는 것이 적절하다.

향후 과제

- 코스피 지수와 시가총액 가중 벤치마크와의 비교
- 지배주주 기준 자본·순이익을 DART 에서 받아 백테스트 전체를 다시 계산
- 주당이익 사용, 업종 중립, 거래비용 민감도
- 버핏식 우량 기업 기준과의 결합(별도 보고서 참고)

부록. 코드 전체

실행 순서와 환경

1. 1_자료수집.py : kor_ticker, kor_price, kor_fs, kor_sector, crawl_log 생성 (KRX, 네이버, FnGuide).
2. 3_장기데이터_수집.py : OpenDART 재무(dart_corp, dart_fs)와 주가 2015 년 이전 보강. [DART-4]는 하루 호출 한도에 걸리면 멈추고 다시 실행하면 이어받는다.
3. 4_상장폐지_수집.py : 상장폐지 종목 목록과 가격(kor_delisted, kor_price_delisted).
4. 2_그레이엄_분석백테스트.py : 이 보고서의 분석. 맨 위 설정에서 FS_SOURCE = 'dart', INCLUDE_DELISTED = True 로 실행했다.

- DB 비밀번호는 코드에 쓰지 않고 환경변수 DB_PW 로 읽거나 실행할 때 입력한다. 윈도우: setx DB_PW "비밀번호" 후 Spyder 재시작.
- OpenDART 인증키는 환경변수 DART_KEY 로 읽는다. KRX 로그인이 필요한 구간은 환경변수 KRX_ID, KRX_PW 를 쓴다.
- 파일은 #%% 셀 단위로 나뉘어 있고 Spyder 에서 셀 단위로 실행한다. 각 파일 맨 위 [안내] 셀에 필요한 패키지와 conda 가상환경(ta_env) 접속 방법이 있다.
- 데이터가 이미 DB 에 있으면 RUN_DOWNLOAD = False 로 두고 분석 파일부터 실행한다. 필요한 테이블이 없으면 어떤 테이블이 없는지 알려주고 멈춘다.

부록 A. 1_자료수집.py

```
#%% [수집-0] 안내
"""
국내 주식 데이터 수집 (차단 대비 버전)   파일명: 1_자료수집.py

[하는 일]
KRX 업종분류 + 개별종목 지표 -> kor_ticker / WICS 섹터 -> kor_sector
네이버 수정주가 -> kor_price / FnGuide 재무제표 -> kor_fs

[실행 순서]
[수집-1] 설정, 테이블 준비 -> [수집-2] 최근 영업일 -> [수집-3]~[수집-6] KRX, WICS
-> [수집-7] 주가 -> [수집-8] 재무 -> [수집-9] 결과 확인

[저장 위치]
1) MySQL stock_db           : 최종 저장
2) 홈폴더\\quant_cache        : KRX, WICS 원본 csv 를 날짜별로 보관 (같은 날 재실행하면 접속 안 함)
3) stock_db.crawl_log 테이블  : 종목별 진행 상황 (중간에 멈춰도 이어받기)

[KRX 로그인]
KRX 데이터 사이트가 로그인을 요구해서 pykrx 도 KRX 회원 아이디/비밀번호가 필요하다.
pykrx 는 환경변수 KRX_ID, KRX_PW 를 읽는다. (윈도우 영구 저장: 명령 프롬프트에서
  setx KRX_ID "아이디"
  setx KRX_PW "비밀번호"
후 Spyder 를 완전히 껐다 켠다.)

[DB 비밀번호]
코드에 직접 쓰지 않는다. 환경변수 DB_PW 에 넣거나 실행할 때 입력한다.
  setx DB_PW "비밀번호"   (이후 Spyder 재시작)

[새 컴퓨터에서]
MySQL 에서 먼저 create database stock_db; 를 한 번 실행한다. 테이블은 [수집-1] 이 만든다.

[패키지] 기존에 쓰던 것과 같다 (pykrx, requests, beautifulsoup4, tqdm, sqlalchemy, pymysql, pandas)
없는 게 있으면 Anaconda Prompt 에서:
  conda activate ta_env
  python -m pip install pykrx beautifulsoup4 tqdm sqlalchemy pymysql
Spyder 에서 ta_env 를 쓰려면 Tools > Preferences > Python interpreter 에서
ta_env 의 python.exe 를 지정하거나, 위처럼 activate 한 뒤 spyder 를 실행한다.

[실행 습관]
- KRX 는 장 마감 후(저녁)에 하루 한 번만. 같은 날 다시 돌려도 캐시를 읽으니 접속하지 않는다.
- 주가/재무 셀은 중간에 멈춰도 된다. 다시 실행하면 한 종목부터 이어서 받는다.
"""


#%% [수집-1] 공통 설정 + 안전 요청 함수 + DB 테이블 준비
import os
import re
import time
import random
import getpass
from io import BytesIO
from datetime import datetime, date, timedelta
from urllib.parse import quote_plus

import numpy as np
import pandas as pd
import requests as rq
import pymysql
from bs4 import BeautifulSoup
from dateutil.relativedelta import relativedelta
from sqlalchemy import create_engine
from tqdm import tqdm

# ---- 설정값 ----
CACHE_DIR = os.path.join(os.path.expanduser('~'), 'quant_cache')   # 원본 csv 보관 폴더
os.makedirs(CACHE_DIR, exist_ok=True)

SLEEP_PRICE = (0.3, 0.8)      # 주가: 종목 사이 랜덤 대기(초)
SLEEP_FS = (1.0, 2.0)         # 재무: 종목 사이 랜덤 대기(초)
MAX_CONSEC_FAIL = 10          # 연속 실패가 이 횟수에 닿으면 차단으로 보고 멈춘다
```

```
PRICE_YEARS = 5               # 주가를 몇 년치 받을지
FS_REFRESH_DAYS = 30          # 재무는 이 기간 안에 받은 종목이면 건너뛴다

DB_USER = 'root'
DB_HOST = '127.0.0.1'
DB_NAME = 'stock_db'
DB_PW = os.environ.get('DB_PW') or getpass.getpass('DB password: ')


def db_connect():
    """읽기용 sqlalchemy 엔진과 쓰기용 pymysql 연결을 같이 만든다."""
    engine = create_engine(
        f'mysql+pymysql://{DB_USER}:{quote_plus(DB_PW)}@{DB_HOST}:3306/{DB_NAME}')
    con = pymysql.connect(user=DB_USER, password=DB_PW, host=DB_HOST,
                          database=DB_NAME, charset='utf8')
    return engine, con


# ---- 차단 대비 도구 ----
class BlockedError(Exception):
    """연속 실패/로그인 풀림 등 차단이 의심될 때"""


class NoData(Exception):
    """응답은 정상인데 그 종목의 데이터가 없는 경우 (신규상장 등). 차단으로 세지 않는다."""


class Breaker:
    """연속 실패 횟수를 세다가 한도에 닿으면 BlockedError 를 낸다."""
    def __init__(self, limit=MAX_CONSEC_FAIL):
        self.n = 0
        self.limit = limit

    def ok(self):
        self.n = 0

    def fail(self):
        self.n += 1
        if self.n >= self.limit:
            raise BlockedError(f'연속 {self.n}번 실패 -> 차단이 의심되어 중단합니다. '
                               '시간을 두고 다시 실행하면 이어서 받습니다.')


def nap(rng):
    """랜덤 대기. 일정한 간격으로 요청하면 기계처럼 보이기 쉬워서 흔든다."""
    time.sleep(random.uniform(*rng))


def safe_call(fn, tries=3, base=3):
    """fn 을 실행하고, 실패하면 3초, 6초... 늘려 가며 재시도한다. 끝까지 실패하면 예외를 그대로 올린다."""
    for k in range(tries):
        try:
            return fn()
        except Exception:
            if k == tries - 1:
                raise
            time.sleep(base * (2 ** k) + random.uniform(0, 1))


def log_done(cur, con, kind, ticker, status):
    """종목별 진행 상황 기록. kind: 'price' / 'fs', status: 'ok' / 'nodata'"""
    cur.execute("""
        insert into crawl_log (대상, 종목코드, 완료일, 상태)
        values (%s,%s,%s,%s) as new
        on duplicate key update 완료일 = new.완료일, 상태 = new.상태
    """, (kind, ticker, date.today().strftime('%Y-%m-%d'), status))
    con.commit()


# ---- 테이블 만들기 (없을 때만) ----
DDL = [
"""
```

```
create table if not exists kor_ticker
(
    종목코드 varchar(6) not null,
    종목명 varchar(20),
    시장구분 varchar(6),
    종가 float,
    시가총액 float,
    기준일 date,
    EPS float,
    PER float,
    BPS float,
    주당배당금 float,
    종목구분 varchar(5),
    primary key(종목코드, 기준일)
);
""",
"""
create table if not exists kor_sector
(
    IDX_CD varchar(3),
    CMP_CD varchar(6),
    CMP_KOR varchar(20),
    SEC_NM_KOR varchar(10),
    기준일 date,
    primary key(CMP_CD, 기준일)
);
""",
"""
create table if not exists kor_price
(
    날짜 date,
    시가 double,
    고가 double,
    저가 double,
    종가 double,
    거래량 double,
    종목코드 varchar(6),
    primary key(날짜, 종목코드)
);
""",
"""
create table if not exists kor_fs
(
    계정 varchar(30),
    기준일 date,
    값 float,
    종목코드 varchar(6),
    공시구분 varchar(1),
    primary key(계정, 기준일, 종목코드, 공시구분)
);
""",
"""
create table if not exists crawl_log
(
    대상 varchar(10),
    종목코드 varchar(6),
    완료일 date,
    상태 varchar(10),
    primary key(대상, 종목코드)
);
""",
]

engine, con = db_connect()
mycursor = con.cursor()
for q in DDL:
    mycursor.execute(q)
con.commit()
engine.dispose()
con.close()
print('테이블 준비 완료')
```

```
#%% [수집-2] 최근 영업일 구하기
"""
- 오늘이 휴일/주말이면 직전 영업일, 영업일이면 오늘을 돌려준다. 결과는 'yyyymmdd' 문자열.
- 하루에 한 번만 KRX 에 물어보고 결과를 캐시 파일에 적어 둔다.
- 장 마감 전에 돌리면 오늘 데이터가 아직 없어서 뒤쪽 셀에서 빈 결과나 오류가 날 수 있다. 마감 후에 실행한다.
"""
def get_last_biz():
    f = os.path.join(CACHE_DIR, f'last_biz_{date.today():%Y%m%d}.txt')
    if os.path.exists(f):
        with open(f) as fp:
            return fp.read().strip()

    from pykrx import stock            # 가져올 때 KRX 로그인이 일어난다 (KRX_ID / KRX_PW 사용)
    today = datetime.today().strftime('%Y%m%d')
    try:
        # 방법 1: pykrx 가 가까운 영업일을 계산
        v = safe_call(lambda: stock.get_nearest_business_day_in_a_week(today))
    except Exception:
        # 방법 2: 삼성전자 일봉의 마지막 날짜. 실제 거래 데이터가 있는 마지막 날이라 가장 확실하다
        start = (datetime.today() - timedelta(days=10)).strftime('%Y%m%d')
        df = safe_call(lambda: stock.get_market_ohlcv(start, today, '005930'))
        v = df.index[-1].strftime('%Y%m%d')

    with open(f, 'w') as fp:
        fp.write(v)
    return v


last_biz = get_last_biz()
print('최근 영업일:', last_biz)


#%% [수집-3] KRX 업종분류 현황 (코스피 / 코스닥)
"""
KRX 정보데이터시스템에서 csv 를 받는 과정은 2단계다.
(1) 원하는 항목을 쿼리로 보내 OTP(generate.cmd)를 받는다.
    -> 개발자 도구(F12) Network 탭의 generate.cmd 에서 request URL 이 제출 주소, payload 가 쿼리 항목이다.
(2) 받은 OTP 를 download.cmd 에 제출하면 csv 가 내려온다.
Referer 를 같이 보내야 한다. (1)의 과정을 거치지 않고 바로 (2)에 보내면 봇으로 본다.

아래 krx_csv() 는 이 두 단계를 묶은 함수다.
- 같은 이름의 캐시 파일이 있으면 KRX 에 접속하지 않고 그 파일을 읽는다.
- OTP 대신 LOGOUT 이나 HTML 이 오면 로그인이 풀렸거나 차단된 것이므로 멈춘다.
- 받은 내용이 HTML 이면 캐시에 저장하지 않는다 (잘못된 파일이 캐시에 남는 것을 막기 위해).
"""
gen_otp_url = 'https://data.krx.co.kr/comm/fileDn/GenerateOTP/generate.cmd'
down_url = 'https://data.krx.co.kr/comm/fileDn/download_csv/download.cmd'
headers = {'Referer': 'https://data.krx.co.kr/contents/MDC/MDI/mdiLoader'}

krx = None             # pykrx 가 로그인해 둔 연결. 캐시가 없어서 접속해야 할 때만 만든다


def krx_csv(query, cache_name, **read_kw):
    global krx
    path = os.path.join(CACHE_DIR, cache_name + '.csv')

    if not os.path.exists(path):
        if krx is None:
            from pykrx.website.comm.auth import get_auth_session
            krx = get_auth_session()

        def _get():
            otp = krx.post(gen_otp_url, headers=headers, data=query, timeout=30).text
            if 'LOGOUT' in otp or '<' in otp:
                raise BlockedError('OTP 를 못 받았습니다 (로그인 풀림 또는 차단 의심): ' + otp[:100])
            res = krx.post(down_url, headers=headers, data={'code': otp}, timeout=60)
            res.raise_for_status()
            if res.content.lstrip()[:1] == b'<':
                raise BlockedError('csv 대신 HTML 이 왔습니다 (차단 의심)')
            return res.content
```

```
        content = safe_call(_get)
        with open(path, 'wb') as f:
            f.write(content)
        time.sleep(random.uniform(1, 2))            # 연속 요청 사이에 잠깐 쉬기

    return pd.read_csv(path, encoding='EUC-KR', **read_kw)   # KRX csv 는 EUC-KR


# 코스피
gen_otp_stk = {
    'mktId': 'STK',            # STK 는 코스피, KSQ 는 코스닥
    'trdDd': last_biz,         # 최근 영업일
    'money': '1',
    'csvxls_isNo': 'false',
    'name': 'fileDown',
    'url': 'dbms/MDC/STAT/standard/MDCSTAT03901'
}
sector_stk = krx_csv(gen_otp_stk, f'krx_sector_STK_{last_biz}', dtype={'종목코드': str})

# 코스닥 (코스피와 같은 코드, mktId 만 KSQ)
gen_otp_ksq = dict(gen_otp_stk, mktId='KSQ')
sector_ksq = krx_csv(gen_otp_ksq, f'krx_sector_KSQ_{last_biz}', dtype={'종목코드': str})

krx_sector = pd.concat([sector_stk, sector_ksq], ignore_index=True)   # 두 데이터 합치기
krx_sector['종목명'] = krx_sector['종목명'].str.strip()                  # 종목명 공백 제거
krx_sector['기준일'] = last_biz

print(sector_stk.shape, sector_ksq.shape, krx_sector.shape)
krx_sector.head()


#%% [수집-4] KRX 개별종목 지표 (PER / PBR / 배당수익률 - [12021])
"""
위와 같은 방식이고 요청하는 쿼리 값만 다르다.
다른 항목이 필요하면 개발자 도구에서 csv 버튼을 눌렀을 때 나오는 generate.cmd 의 payload 를 보고 맞춘다.
"""
gen_otp_ind = {
    'searchType': '1',
    'mktId': 'ALL',
    'trdDd': last_biz,
    'csvxls_isNo': 'false',
    'name': 'fileDown',
    'url': 'dbms/MDC/STAT/standard/MDCSTAT03501'
}
krx_ind = krx_csv(gen_otp_ind, f'krx_ind_{last_biz}', dtype={'종목코드': str})
krx_ind['종목명'] = krx_ind['종목명'].str.strip()
krx_ind['기준일'] = last_biz

krx_ind.head()


#%% [수집-5] 데이터 정리 + kor_ticker 저장
diff = list(set(krx_sector['종목명']).symmetric_difference(set(krx_ind['종목명'])))
print(diff)
# 하나의 데이터에만 있는 종목이다. 리츠, 선박/광물 펀드, 해외종목처럼 일반적이지 않은 경우가 많다.
# 그래서 두 데이터를 합친 뒤 '기타'로 구분한다.

kor_ticker = pd.merge(krx_sector,
                      krx_ind,
                      on=krx_sector.columns.intersection(krx_ind.columns).tolist(),
                      how='outer')
# 두 표에 공통으로 있는 열(종목코드, 종목명, 종가, 대비, 등락률 등)을 기준으로 합친다.

# 스팩: '스팩' 이 들어가거나 '제n호' 형태. 우선주: 종목코드 끝자리가 0 이 아님. 리츠: 이름이 리츠로 끝남(중간에 들어간 경우 제외)
kor_ticker['종목구분'] = np.where(kor_ticker['종목명'].str.contains('스팩|제[0-9]+호'), '스팩',
                             np.where(kor_ticker['종목코드'].str[-1:] != '0', '우선주',
                                      np.where(kor_ticker['종목명'].str.endswith('리츠'), '리츠',
                                               np.where(kor_ticker['종목명'].isin(diff), '기타',
                                                        '보통주'))))

kor_ticker = kor_ticker.reset_index(drop=True)
kor_ticker.columns = kor_ticker.columns.str.replace(' ', '')       # 열 이름 공백 삭제
```

```
kor_ticker = kor_ticker[['종목코드', '종목명', '시장구분', '종가', '시가총액', '기준일', 'EPS',
                         'PER', 'BPS', '주당배당금', '종목구분']]
kor_ticker = kor_ticker.replace({np.nan: None})                    # SQL 에는 NaN 이 안 들어가므로 None 으로
kor_ticker['기준일'] = pd.to_datetime(kor_ticker['기준일'])          # yyyymmdd -> yyyy-mm-dd

print(kor_ticker['종목구분'].value_counts())

engine, con = db_connect()
mycursor = con.cursor()

query = """
    insert into kor_ticker(종목코드,종목명,시장구분,종가,시가총액,기준일,EPS,PER,BPS,주당배당금,종목구분)
    values(%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s) as new
    on duplicate key update
    종목명 = new.종목명, 시장구분 = new.시장구분, 종가 = new.종가, 시가총액 = new.시가총액,
    EPS = new.EPS, PER = new.PER, BPS = new.BPS, 주당배당금 = new.주당배당금, 종목구분 = new.종목구분;
"""
mycursor.executemany(query, kor_ticker.values.tolist())
con.commit()

engine.dispose()
con.close()
print('kor_ticker 저장 완료:', len(kor_ticker))


#%% [수집-6] WICS 기준 섹터 정보
"""
GICS 는 KRX 가 공개하지만 독점적 지적재산이라 쓰기 곤란하다. 그래서 FnGuide 의 WICS 를 쓴다.
페이지: http://www.wiseindex.com/Index  (WISE Sector Index -> WICS -> 섹터 선택 -> Components)
개발자 도구 Network 탭의 getindexcomponent 요청이 데이터 주소다.
  https://www.wiseindex.com/Index/GetIndexComponets?ceil_yn=0&dt=20261008&sec_cd=G10
  ceil_yn=0 : 실링 여부(0 은 비실링) / dt : 조회일자 / sec_cd : 섹터 코드
결과가 JSON 이라 requests 의 json() 으로 바로 받을 수 있다.
날짜는 위에서 구한 last_biz 를 쓴다. 장이 끝나기 전에는 데이터가 없어서 오류가 날 수 있다.
같은 날 다시 실행하면 캐시 파일을 읽는다.
"""
def get_wics(base_date):
    path = os.path.join(CACHE_DIR, f'wics_{base_date}.csv')
    if os.path.exists(path):
        return pd.read_csv(path, dtype={'IDX_CD': str, 'CMP_CD': str})

    sector_code = ['G25', 'G35', 'G50', 'G40', 'G10', 'G20', 'G55', 'G30', 'G15', 'G45']
    data_sector = []                                    # 섹터 정보가 들어갈 빈 리스트

    for i in tqdm(sector_code):
        url = f'https://www.wiseindex.com/Index/GetIndexComponets?ceil_yn=0&dt={base_date}&sec_cd={i}'
        data = safe_call(lambda: rq.get(url, timeout=30).json())
        data_sector.append(pd.json_normalize(data['list']))
        nap((1.5, 2.5))

    sector = pd.concat(data_sector, axis=0)
    if sector.empty:
        raise ValueError('WICS 데이터가 비어 있습니다 (장 마감 전이거나 날짜 문제)')
    sector = sector[['IDX_CD', 'CMP_CD', 'CMP_KOR', 'SEC_NM_KOR']].copy()
    sector.to_csv(path, index=False, encoding='utf-8-sig')
    return sector


sector = get_wics(last_biz)
sector['기준일'] = pd.to_datetime(last_biz)

engine, con = db_connect()
mycursor = con.cursor()

query = """
    insert into kor_sector (IDX_CD, CMP_CD, CMP_KOR, SEC_NM_KOR, 기준일)
    values(%s,%s,%s,%s,%s) as new
    on duplicate key update
    IDX_CD = new.IDX_CD, CMP_KOR = new.CMP_KOR, SEC_NM_KOR = new.SEC_NM_KOR
"""
mycursor.executemany(query, sector.values.tolist())
con.commit()
```

```
engine.dispose()
con.close()
print('kor_sector 저장 완료:', len(sector))


#%% [수집-7] 수정주가 (증분 + 이어받기)
"""
주가는 퀀트 투자와 백테스트에 꼭 필요하고, 백테스트에는 수정주가가 필요하다. (네이버 차트 사용)

처음에는 PRICE_YEARS 년치를 받고, 그 뒤로는 DB 의 마지막 날짜 이후만 받는다.
수정주가는 액면분할/배당 등이 생기면 과거 값이 바뀐다. 그래서 마지막 날짜보다 10일 앞부터 받아서
DB 에 있는 값과 비교하고, 다르면 그 종목만 전체 기간을 다시 받는다.
오늘 이미 처리한 종목은 crawl_log 를 보고 건너뛴다. 중간에 멈췄다가 다시 실행하면 이어서 받는다.
"""
engine, con = db_connect()
mycursor = con.cursor()

# 티커 리스트 (최근일 기준 보통주)
ticker_list = pd.read_sql("""
select * from kor_ticker
where 기준일 = (select max(기준일) from kor_ticker)
    and 종목구분 = '보통주';
""", con=engine)

today_str = date.today().strftime('%Y-%m-%d')
fr_full = (date.today() + relativedelta(years=-PRICE_YEARS)).strftime('%Y%m%d')   # 시작일
to = date.today().strftime('%Y%m%d')                                              # 종료일

# 오늘 이미 처리한 종목
done_today = set(pd.read_sql(f"""
select 종목코드 from crawl_log where 대상 = 'price' and 완료일 = '{today_str}';
""", con=engine)['종목코드'])

# 종목별 DB 의 마지막 날짜
last_map = pd.read_sql("""
select 종목코드, max(날짜) as 마지막 from kor_price group by 종목코드;
""", con=engine)
last_map['마지막'] = pd.to_datetime(last_map['마지막'])
last_map = last_map.set_index('종목코드')['마지막'].to_dict()

# 수정주가 비교용: 최근 40일 종가
recent = pd.read_sql(f"""
select 종목코드, 날짜, 종가 from kor_price
where 날짜 >= '{(date.today() - timedelta(days=40)).strftime('%Y-%m-%d')}';
""", con=engine)
recent['날짜'] = pd.to_datetime(recent['날짜'])
recent_map = recent.set_index(['종목코드', '날짜'])['종가'].to_dict()


def fetch_price(ticker, fr, to):
    """네이버 차트에서 일봉을 받아 정리한다. 결과 열: 날짜, 시가, 고가, 저가, 종가, 거래량, 종목코드"""
    url = f'https://fchart.stock.naver.com/siseJson.nhn?
symbol={ticker}&requestType=1&startTime={fr}&endTime={to}&timeframe=day'

    def _get():
        r = rq.get(url, timeout=20)
        r.raise_for_status()
        return r.content

    data = safe_call(_get)
    data_price = pd.read_csv(BytesIO(data))

    price = data_price.iloc[:, 0:6].copy()                      # 날짜, 가격, 거래량 해당 열만
    price.columns = ['날짜', '시가', '고가', '저가', '종가', '거래량']
    price = price.dropna()
    price['날짜'] = price['날짜'].str.extract(r'(\d+)')          # 날짜 열에서 숫자만 추출
    price['날짜'] = pd.to_datetime(price['날짜'])
    price['종목코드'] = ticker
    if len(price) == 0:
        raise NoData(ticker)
    return price
```

```
query_price = """
    insert into kor_price(날짜, 시가, 고가, 저가, 종가, 거래량, 종목코드)
    values(%s,%s,%s,%s,%s,%s,%s) as new
    on duplicate key update
    시가 = new.시가, 고가 = new.고가, 저가 = new.저가,
    종가 = new.종가, 거래량 = new.거래량;
"""

todo = [t for t in ticker_list['종목코드'] if t not in done_today]
print(f'대상 {len(ticker_list)}개 중 오늘 처리 완료 {len(done_today)}개 -> 이번에 받을 종목 {len(todo)}개')

error_list = []
br = Breaker()

try:
    for ticker in tqdm(todo):
        try:
            last = last_map.get(ticker)

            if last is None:                                            # DB 에 처음 들어오는 종목
                price = fetch_price(ticker, fr_full, to)
            else:
                fr = (last - timedelta(days=10)).strftime('%Y%m%d')
                price = fetch_price(ticker, fr, to)

                # 겹치는 구간의 종가가 DB 와 다르면 수정주가가 바뀐 것이므로 전체 기간을 다시 받는다
                changed = False
                ov = price[price['날짜'] <= last]
                for d, c in zip(ov['날짜'], ov['종가']):
                    old = recent_map.get((ticker, d))
                    if old is not None and abs(old - c) > abs(old) * 0.001:
                        changed = True
                        break
                if changed:
                    nap(SLEEP_PRICE)
                    price = fetch_price(ticker, fr_full, to)

            mycursor.executemany(query_price, price.values.tolist())
            con.commit()
            log_done(mycursor, con, 'price', ticker, 'ok')
            br.ok()

        except BlockedError:
            raise
        except NoData:
            log_done(mycursor, con, 'price', ticker, 'nodata')         # 데이터가 없는 종목은 오늘 다시 시도하지 않는다
        except Exception as e:
            print(ticker, type(e).__name__, str(e)[:80])
            error_list.append(ticker)
            br.fail()                                                  # 연속 실패가 쌓이면 BlockedError

        nap(SLEEP_PRICE)

except BlockedError as e:
    print('중단:', e)

finally:
    engine.dispose()
    con.close()

print('실패 종목 수:', len(error_list))


#%% [수집-8] 재무제표 (FnGuide, 이어받기)
"""
가치투자의 핵심 데이터다. FnGuide 의 Company Guide (http://comp.fnguide.com/) 에서 받는다.
재무제표 탭에는 포괄손익계산서, 재무상태표, 현금흐름표가 있고, 주소 뒤쪽은 필요 없어서 티커만 바꿔 쓴다.

최근 FS_REFRESH_DAYS 일 안에 받은 종목은 crawl_log 를 보고 건너뛴다.
전 종목을 처음 받으면 몇 시간 걸린다. 중간에 멈춰도(Ctrl+C) 다시 실행하면 이어서 받는다.
```

```
"""
fnguide_headers = {'User-Agent': 'Mozilla/5.0'}


def fnguide_fs(name, ticker, freq):
    """name: getFinIncome(손익) / getFinBalance(재무상태) / getFinCashFlow(현금흐름), freq: 'Y'=연간, 'Q'=분기
       결과: 첫 열 'IFRS(연결)' + 기간 열들"""
    nap((0.2, 0.5))                                          # 한 종목에 6번 요청하므로 요청 사이에도 잠깐 쉰다
    u = f'https://wcomp.fnguide.com/CompanyInfo/{name}'

    def _get():
        r = rq.get(u, params={'cmp_cd': ticker, 'freq_typ': freq, 'consol_typ': 'C'},
                   headers=fnguide_headers, timeout=20)
        r.raise_for_status()
        return r.json()['dataset']

    o = safe_call(_get)

    if isinstance(o, list) and len(o) > 0 and 'data' in o[0]:
        o = o[0]

    if not (isinstance(o, dict) and 'data' in o and 'header' in o):   # 구조가 다르면 보여주고 멈춤
        print(type(o), (list(o.keys()) if isinstance(o, dict) else str(o)[:300]))
        raise ValueError(f'{name} {freq}: 구조 확인 필요')

    df = pd.DataFrame(o['data'])
    # VAL1, VAL2... -> 2023/12 ...  (일부 신규상장 종목은 YYMM 이 None 인 열이 있어서 그런 열은 버린다)
    rename = {h['CD']: str(h['YYMM']).replace(' (최근분기)', '')
              for h in o['header'] if h.get('YYMM')}
    if not rename:
        raise NoData(ticker)
    df = df.rename(columns=rename)
    df['NAME'] = df['NAME'].str.strip()
    df = df[['NAME'] + list(rename.values())]
    df = df.rename(columns={'NAME': 'IFRS(연결)'})
    for c in rename.values():
        df[c] = pd.to_numeric(df[c], errors='coerce')      # 숫자 변환 (글자는 NaN)
    return df


def fnguide_tables(ticker):
    """표 10.1 순서대로 6개: 0 손익(연간) 1 손익(분기) 2 재무상태(연간) 3 재무상태(분기) 4 현금흐름(연간) 5 현금흐름(분기)"""
    return [fnguide_fs('getFinIncome', ticker, 'Y'),
            fnguide_fs('getFinIncome', ticker, 'Q'),
            fnguide_fs('getFinBalance', ticker, 'Y'),
            fnguide_fs('getFinBalance', ticker, 'Q'),
            fnguide_fs('getFinCashFlow', ticker, 'Y'),
            fnguide_fs('getFinCashFlow', ticker, 'Q')]


def clean_fs(df, ticker, frequency):
    """입력: 데이터프레임, 티커, 공시구분(연간/분기)"""
    df = df[~df.loc[:, ~df.columns.isin(['계정'])].isna().all(axis=1)]         # 모든 연도가 NaN 인 항목 제외
    df = df.drop_duplicates(['계정'], keep='first')                            # 계정명이 중복되면 첫 번째만
    df = pd.melt(df, id_vars='계정', var_name='기준일', value_name='값')         # 열로 긴 데이터를 행으로 긴 데이터로
    df = df[~pd.isnull(df['값'])]                                              # 계정값이 없는 항목 제외
    df['계정'] = df['계정'].replace({'계산에 참여한 계정 펼치기': ''}, regex=True)   # 페이지의 [+] 부분 글자 제거
    df['기준일'] = pd.to_datetime(df['기준일'],
                                  format='%Y/%m') + pd.tseries.offsets.MonthEnd()  # 월말 일자를 붙인다
                                                   # (책은 format='%Y-%m' 이지만 이 페이지의 열 이름은 '2023/12' 형태라
'%Y/%m')
    df['종목코드'] = ticker
    df['공시구분'] = frequency
    return df


def collect_fs(ticker):
    """한 종목의 연간 + 분기 재무제표를 정리해서 돌려준다."""
    url = f'https://wcomp.fnguide.com/CompanyInfo/Finance?c_id=AA&menu_type=01&cmp_cd={ticker}'

    data = fnguide_tables(ticker)
```

```
    # 연간 데이터 (포괄손익계산서에서는 '전년동기' 열을 뺀다)
    data_fs_y = pd.concat([
        data[0].iloc[:, ~data[0].columns.str.contains('전년동기')], data[2], data[4]])
    data_fs_y = data_fs_y.rename(columns={data_fs_y.columns[0]: '계정'})

    # 결산월 찾기: 결산 전에 크롤링하면 연간 표에 분기 열이 섞여 들어오므로 결산월 열만 남긴다
    page_data = safe_call(lambda: rq.get(url, headers=fnguide_headers, timeout=20))
    page_data_html = BeautifulSoup(page_data.content, 'html.parser')

    fiscal_data = page_data_html.select('div.corp_group1 > h2')
    if len(fiscal_data) > 1:
        fiscal_data_text = re.findall('[0-9]+', fiscal_data[1].text)    # 'n월 결산' 에서 숫자만
    else:                                                                # (예비) 재무상태표(연간)의 'yyyy/mm' 열에서 월만 뽑기
        fiscal_data_text = list({c[-2:] for c in data[2].columns[1:] if re.fullmatch(r'\d{4}/\d{2}', str(c))})

    data_fs_y = data_fs_y.loc[:, (data_fs_y.columns == '계정') | (
        data_fs_y.columns.str[-2:].isin(fiscal_data_text))]
    data_fs_y_clean = clean_fs(data_fs_y, ticker, 'y')

    # 분기 데이터 (결산월 선택은 필요 없다)
    data_fs_q = pd.concat([
        data[1].iloc[:, ~data[1].columns.str.contains('전년동기')], data[3], data[5]])
    data_fs_q = data_fs_q.rename(columns={data_fs_q.columns[0]: '계정'})
    data_fs_q_clean = clean_fs(data_fs_q, ticker, 'q')

    data_fs_bind = pd.concat([data_fs_y_clean, data_fs_q_clean])
    if len(data_fs_bind) == 0:
        raise NoData(ticker)
    return data_fs_bind


engine, con = db_connect()
mycursor = con.cursor()

# 티커 리스트 (시가총액 큰 종목부터. 중간에 멈춰도 큰 종목은 먼저 들어와 있다)
ticker_list = pd.read_sql("""
select * from kor_ticker
where 기준일 = (select max(기준일) from kor_ticker)
    and 종목구분 = '보통주'
order by 시가총액 desc;
""", con=engine)

# 최근 FS_REFRESH_DAYS 일 안에 정상 수집한 종목 / 데이터가 없다고 확인된 종목
since = (date.today() - timedelta(days=FS_REFRESH_DAYS)).strftime('%Y-%m-%d')
done_fs = set(pd.read_sql(f"""
select 종목코드 from crawl_log
where 대상 = 'fs' and 완료일 >= '{since}';
""", con=engine)['종목코드'])

query = """
    insert into kor_fs (계정, 기준일, 값, 종목코드, 공시구분)
    values (%s,%s,%s,%s,%s) as new
    on duplicate key update
    값=new.값
"""

todo = [t for t in ticker_list['종목코드'] if t not in done_fs]
print(f'대상 {len(ticker_list)}개 중 최근 {FS_REFRESH_DAYS}일 내 처리 {len(done_fs)}개 -> 이번에 받을 종목 {len(todo)}개')

error_list = []
br = Breaker()

try:
    for ticker in tqdm(todo):
        try:
            data_fs_bind = collect_fs(ticker)

            mycursor.executemany(query, data_fs_bind.values.tolist())
            con.commit()
            log_done(mycursor, con, 'fs', ticker, 'ok')
            br.ok()
```

```
        except BlockedError:
            raise
        except NoData:
            log_done(mycursor, con, 'fs', ticker, 'nodata')
        except Exception as e:
            print(ticker, type(e).__name__, str(e)[:80])
            error_list.append(ticker)
            br.fail()

        nap(SLEEP_FS)

except BlockedError as e:
    print('중단:', e)

finally:
    engine.dispose()
    con.close()

print('실패 종목 수:', len(error_list))
# 시간이 지나 다시 실행하면 upsert 라서 수정된 재무는 update, 새로 나온 재무는 insert 된다.


#%% [수집-9] 수집 결과 확인
engine, con = db_connect()

print(pd.read_sql("""
select 'kor_ticker' as 테이블, count(*) as 행수, max(기준일) as 최근일 from kor_ticker
union all
select 'kor_sector', count(*), max(기준일) from kor_sector
union all
select 'kor_price', count(*), max(날짜) from kor_price
union all
select 'kor_fs', count(*), max(기준일) from kor_fs;
""", con=engine))

print(pd.read_sql("""
select 대상, 상태, count(*) as 종목수 from crawl_log group by 대상, 상태;
""", con=engine))

engine.dispose()
con.close()
```

부록 B. 3_장기데이터_수집.py

```
#%% [DART-0] 안내
"""
장기 데이터 수집: OpenDART 재무제표(2015년 이후) + 주가 과거 보강   파일명: 3_장기데이터_수집.py

[왜 필요한가]
FnGuide 페이지는 연간 재무가 4~5년치뿐이라 그레이엄 엄격 버전(10년 흑자/성장)이 계산되지 않는다.
OpenDART 는 2015년 이후 사업보고서를 주고, 사업보고서 하나에 당기/전기/전전기 3개 연도가 들어 있다.
그래서 3년 간격으로 요청하면 2013년분까지 얻을 수 있다. (예: 2025, 2022, 2019, 2016 보고서 + 2015 보고서)

[실행 전에]
1) https://opendart.fss.or.kr 에서 회원가입 후 인증키(40자리)를 신청한다.
2) 명령 프롬프트에서  setx DART_KEY "인증키"  후 Spyder 를 완전히 껐다 켠다. (또는 실행할 때 입력)
3) 1_자료수집.py 의 [수집-5] 까지 끝나 kor_ticker 가 있어야 한다.
   상장폐지 종목도 같이 받으려면 4_상장폐지_수집.py 의 [상폐-3] 까지 먼저 실행한다.

[저장 위치]
- dart_fs   : 종목코드, 계정(짧은 이름), 기준일, 값(억원)   -> 2_그레이엄 파일의 FS_SOURCE = 'dart' 가 읽는다
- dart_corp : 종목코드 <-> OpenDART 고유번호(corp_code), 결산월
- crawl_log : 종목별 진행 상황 (중간에 멈춰도 이어받기)

[공식 문서(opendart.fss.or.kr 가이드)로 확인한 것]
- fnlttSinglAcntAll.json : 인자 crtfc_key, corp_code, bsns_year, reprt_code(사업보고서 11011), fs_div(CFS 연결 / OFS
별도)
- bsns_year 는 2015년 이후부터 제공
- 응답 항목 : sj_div(BS, IS, CIS, CF, SCE), account_id, account_nm, thstrm_amount, frmtrm_amount, bfefrmtrm_amount
  (bfefrmtrm = 전전기, 사업보고서에만 나옴)
- 상태코드 : 000 정상 / 013 데이터 없음 / 020 요청 제한 초과(보통 2만 건 이상) / 010,011,012,901 키 문제
- corpCode.xml 은 zip 으로 오고 corp_code, corp_name, stock_code, modify_date 가 들어 있다
- company.json 에 acc_mt(결산월) 가 있다

[확인 안 된 것]
- 하루 호출 한도의 정확한 기준 (20,000건은 문서의 오류 설명에서 가져온 숫자)
- corpCode.xml 에 상장폐지 기업이 들어 있는지, 폐지 기업의 stock_code 가 남아 있는지
- 아래 MAP 의 account_id (IFRS 표준 이름을 기억해서 쓴 것). 회사마다 다르게 쓸 수 있어서 [DART-5] 로 커버리지를 확인한다
- 12월 결산이 아닌 회사의 bsns_year 가 어느 연도를 가리키는지. 그래서 12월 결산 회사만 받는다
- 현금흐름표에서 CAPEX 의 부호. 절댓값으로 저장한다
"""


#%% [DART-1] 설정 + 공통 도구 + 테이블 준비
import os
import io
import re
import time
import random
import getpass
import zipfile
import xml.etree.ElementTree as ET
from datetime import date, datetime, timedelta
from urllib.parse import quote_plus
from io import BytesIO

import numpy as np
import pandas as pd
import requests as rq
import pymysql
from dateutil.relativedelta import relativedelta
from sqlalchemy import create_engine, inspect
from tqdm import tqdm

CACHE_DIR = os.path.join(os.path.expanduser('~'), 'quant_cache')
os.makedirs(CACHE_DIR, exist_ok=True)

FIRST_YEAR = 2015            # OpenDART 가 재무제표를 주는 첫 사업연도
DAILY_LIMIT = 19000          # 하루에 보낼 최대 호출 수(코드가 스스로 멈추는 선). 서버 한도는 확인 안 됨
SLEEP_DART = (0.1, 0.3)
MAX_CONSEC_FAIL = 10
DART_REFRESH_DAYS = 120      # 이 기간 안에 받은 종목은 건너뛴다 (연간 재무는 1년에 한 번 바뀐다)

DB_USER = 'root'
```

```
DB_HOST = '127.0.0.1'
DB_NAME = 'stock_db'
DB_PW = os.environ.get('DB_PW') or getpass.getpass('DB password: ')
DART_KEY = os.environ.get('DART_KEY') or getpass.getpass('OpenDART 인증키: ')


def db_connect():
    engine = create_engine(
        f'mysql+pymysql://{DB_USER}:{quote_plus(DB_PW)}@{DB_HOST}:3306/{DB_NAME}')
    con = pymysql.connect(user=DB_USER, password=DB_PW, host=DB_HOST, database=DB_NAME, charset='utf8')
    return engine, con


class BlockedError(Exception):
    """연속 실패 등 차단이 의심될 때"""


class NoData(Exception):
    """응답은 정상인데 데이터가 없는 경우"""


class NonDecember(Exception):
    """12월 결산이 아닌 회사"""


class DartLimit(Exception):
    """하루 호출 한도에 닿았을 때"""


class DartFatal(Exception):
    """인증키/접근 문제처럼 계속 시도해도 소용없는 오류"""


class Breaker:
    def __init__(self, limit=MAX_CONSEC_FAIL):
        self.n = 0
        self.limit = limit

    def ok(self):
        self.n = 0

    def fail(self):
        self.n += 1
        if self.n >= self.limit:
            raise BlockedError(f'연속 {self.n}번 실패 -> 중단합니다. 시간을 두고 다시 실행하면 이어서 받습니다.')


def nap(rng):
    time.sleep(random.uniform(*rng))


def safe_call(fn, tries=3, base=3):
    for k in range(tries):
        try:
            return fn()
        except Exception:
            if k == tries - 1:
                raise
            time.sleep(base * (2 ** k) + random.uniform(0, 1))


def log_done(cur, con, kind, ticker, status):
    cur.execute("""
        insert into crawl_log (대상, 종목코드, 완료일, 상태)
        values (%s,%s,%s,%s) as new
        on duplicate key update 완료일 = new.완료일, 상태 = new.상태
    """, (kind, ticker, date.today().strftime('%Y-%m-%d'), status))
    con.commit()


# ---- 오늘 보낸 호출 수를 파일에 적어 둔다 (같은 날 다시 실행해도 이어서 센다) ----
CALL_FILE = os.path.join(CACHE_DIR, f'dart_calls_{date.today():%Y%m%d}.txt')
```

```
def read_calls():
    try:
        with open(CALL_FILE) as f:
            return int(f.read().strip())
    except Exception:
        return 0


n_calls = read_calls()
FATAL_STATUS = {'010', '011', '012', '101', '800', '901'}


def dart_json(name, **params):
    """OpenDART JSON 호출. 정상(000)이면 dict, 데이터 없음(013)이면 None."""
    global n_calls
    if n_calls >= DAILY_LIMIT:
        raise DartLimit(f'오늘 호출 {n_calls}건에 닿아 멈춥니다. 내일 같은 셀을 다시 실행하면 이어받습니다.')

    def _get():
        r = rq.get(f'https://opendart.fss.or.kr/api/{name}.json',
                   params=dict(crtfc_key=DART_KEY, **params), timeout=30)
        r.raise_for_status()
        return r.json()

    js = safe_call(_get)
    n_calls += 1
    with open(CALL_FILE, 'w') as f:
        f.write(str(n_calls))
    nap(SLEEP_DART)

    st = js.get('status')
    if st == '000':
        return js
    if st == '013':
        return None
    if st == '020':
        raise DartLimit('요청 제한 초과(020). 내일 다시 실행하세요: ' + str(js.get('message')))
    if st in FATAL_STATUS:
        raise DartFatal(f'{st}: {js.get("message")}')
    raise RuntimeError(f'DART 오류 {st}: {js.get("message")}')


DDL = [
"""
create table if not exists dart_corp
(
    corp_code varchar(8) not null,
    종목코드 varchar(6),
    corp_name varchar(100),
    acc_mt varchar(2),
    primary key(corp_code)
);
""",
"""
create table if not exists dart_fs
(
    종목코드 varchar(6) not null,
    계정 varchar(20) not null,
    기준일 date not null,
    값 double,
    primary key(종목코드, 계정, 기준일)
);
""",
"""
create table if not exists crawl_log
(
    대상 varchar(10),
    종목코드 varchar(6),
    완료일 date,
    상태 varchar(10),
```

```
    primary key(대상, 종목코드)
);
""",
]

engine, con = db_connect()
mycursor = con.cursor()
for q in DDL:
    mycursor.execute(q)
con.commit()
engine.dispose()
con.close()
print('테이블 준비 완료 / 오늘 보낸 호출 수:', n_calls)


#%% [DART-2] 종목코드 <-> 고유번호(corp_code) 매핑
"""
corpCode.xml 은 zip 파일로 온다. 압축 안의 xml 에서 corp_code, corp_name, stock_code 를 읽는다.
안의 태그 구조는 문서에 자세히 없어서, corp_code 태그를 가진 요소를 모두 한 행으로 읽게 했다.
"""
def load_corp_codes():
    path = os.path.join(CACHE_DIR, f'dart_corp_{date.today():%Y%m%d}.xml')
    if not os.path.exists(path):
        r = safe_call(lambda: rq.get('https://opendart.fss.or.kr/api/corpCode.xml',
                                     params={'crtfc_key': DART_KEY}, timeout=60))
        r.raise_for_status()
        try:
            z = zipfile.ZipFile(io.BytesIO(r.content))
        except zipfile.BadZipFile:
            raise RuntimeError('corpCode 응답이 zip 이 아닙니다 (인증키 문제일 수 있음): ' + r.text[:200])
        with open(path, 'wb') as f:
            f.write(z.read(z.namelist()[0]))

    rows = []
    for e in ET.parse(path).getroot().iter():
        if e.find('corp_code') is not None:
            rows.append({k: (e.findtext(k) or '').strip()
                         for k in ['corp_code', 'corp_name', 'stock_code', 'modify_date']})
    return pd.DataFrame(rows)


corp_all = load_corp_codes()
print('corpCode 전체 행:', len(corp_all))

corp = corp_all[corp_all['stock_code'].str.len() == 6].copy()
corp = corp.sort_values('modify_date').drop_duplicates('stock_code', keep='last')   # 같은 종목코드면 최신 것
print('종목코드가 있는 회사:', len(corp))

engine, con = db_connect()
mycursor = con.cursor()

query = """
    insert into dart_corp (corp_code, 종목코드, corp_name)
    values (%s,%s,%s) as new
    on duplicate key update 종목코드 = new.종목코드, corp_name = new.corp_name
"""
mycursor.executemany(query, corp[['corp_code', 'stock_code', 'corp_name']].values.tolist())
con.commit()

# 우리 종목이 얼마나 매핑되는지
tk = pd.read_sql("""
select 종목코드 from kor_ticker
where 기준일 = (select max(기준일) from kor_ticker) and 종목구분 = '보통주';
""", con=engine)
print(f'kor_ticker 보통주 {len(tk)}개 중 고유번호가 있는 종목: {tk["종목코드"].isin(corp["stock_code"]).sum()}개')

if 'kor_delisted' in inspect(engine).get_table_names():
    dl = pd.read_sql('select 종목코드 from kor_delisted;', con=engine)
    n_ok = dl['종목코드'].isin(corp['stock_code']).sum()
    print(f'kor_delisted {len(dl)}개 중 고유번호가 있는 종목: {n_ok}개')
    print('-> 0 에 가까우면 corpCode.xml 에 상장폐지 기업의 종목코드가 없다는 뜻이라 폐지 종목 재무는 받을 수 없습니다.')
```

```
engine.dispose()
con.close()


#%% [DART-3] 계정 매핑 + 파싱 함수
"""
이 코드가 쓰는 짧은 계정 이름 -> (재무제표 구분, XBRL 표준계정ID 후보, 계정명 후보)
먼저 표준계정ID 로 찾고, 없으면 계정명으로 찾는다. ID 는 IFRS 이름을 기억해 쓴 것이라 확인이 필요하다.
"""
MAP = {
    'sales':  (('IS', 'CIS'), ['ifrs-full_Revenue'],
               ['매출액', '수익(매출액)', '영업수익', '매출']),
    'op':     (('IS', 'CIS'), ['dart_OperatingIncomeLoss'],
               ['영업이익', '영업이익(손실)']),
    'ni':     (('IS', 'CIS'), ['ifrs-full_ProfitLoss'],
               ['당기순이익', '당기순이익(손실)', '연결당기순이익', '당기순손익']),
    'assets': (('BS',), ['ifrs-full_Assets'], ['자산총계']),
    'liab':   (('BS',), ['ifrs-full_Liabilities'], ['부채총계']),
    'equity': (('BS',), ['ifrs-full_Equity'], ['자본총계']),
    'ca':     (('BS',), ['ifrs-full_CurrentAssets'], ['유동자산']),
    'cl':     (('BS',), ['ifrs-full_CurrentLiabilities'], ['유동부채']),
    'ltd':    (('BS',), ['ifrs-full_NoncurrentLiabilities'], ['비유동부채']),
    'cfo':    (('CF',), ['ifrs-full_CashFlowsFromUsedInOperatingActivities'],
               ['영업활동현금흐름', '영업활동으로인한현금흐름', '영업활동순현금흐름']),
    'capex':  (('CF',), ['ifrs-full_PurchaseOfPropertyPlantAndEquipmentClassifiedAsInvestingActivities',
                         'ifrs-full_PurchaseOfPropertyPlantAndEquipment'],
               ['유형자산의취득', '유형자산취득']),
}


def norm(s):
    return str(s).replace(' ', '').replace(' ', '')


AMT_COLS = ('thstrm_amount', 'frmtrm_amount', 'bfefrmtrm_amount')


def to_num(v):
    """'9,999,999,999' 같은 문자열을 숫자로. 비어 있거나 '-' 면 NaN, '(1,000)' 은 음수로 본다."""
    if v is None:
        return np.nan
    s = str(v).replace(',', '').strip()
    if s in ('', '-'):
        return np.nan
    neg = s.startswith('(') and s.endswith(')')
    if neg:
        s = s[1:-1]
    try:
        return -float(s) if neg else float(s)
    except ValueError:
        return np.nan


def has_value(it):
    return any(not np.isnan(to_num(it.get(c))) for c in AMT_COLS)


def pick_item(items, stmts, ids, names):
    """재무제표 구분 순서대로, 표준계정ID -> 계정명 순으로 찾는다. 금액이 하나라도 있는 첫 번째 항목을 쓴다."""
    names = {norm(n) for n in names}
    for st in stmts:
        for it in items:
            if it.get('sj_div') == st and it.get('account_id') in ids and has_value(it):
                return it
    for st in stmts:
        for it in items:
            if it.get('sj_div') == st and norm(it.get('account_nm', '')) in names and has_value(it):
                return it
    return None


def parse_report(items, bsns_year):
```

```
    """사업보고서 하나에서 당기/전기/전전기 3개 연도의 값을 꺼낸다. 금액은 억원으로 바꾼다."""
    out = []
    for key, (stmts, ids, names) in MAP.items():
        it = pick_item(items, stmts, ids, names)
        if it is None:
            continue
        for col, back in (('thstrm_amount', 0), ('frmtrm_amount', 1), ('bfefrmtrm_amount', 2)):
            v = to_num(it.get(col))
            if np.isnan(v):
                continue
            if key == 'capex':
                v = abs(v)                      # 부호 표기가 확인되지 않아 절댓값으로 저장
            out.append((key, date(int(bsns_year) - back, 12, 31), v / 100000000))
    return pd.DataFrame(out, columns=['계정', '기준일', '값'])


def request_years():
    """3년 간격으로 요청할 사업연도. 마지막에 FIRST_YEAR 를 붙여 2013년분까지 채운다."""
    today = date.today()
    end = today.year - 1 if today.month >= 4 else today.year - 2
    ys = list(range(end, FIRST_YEAR - 1, -3))
    if ys and ys[-1] - 2 > FIRST_YEAR - 2:
        ys.append(FIRST_YEAR)
    return ys


print('요청할 사업연도:', request_years())


#%% [DART-4] 전 종목 재무 수집 (이어받기)
"""
종목당 호출: 기업개황 1번(결산월) + 사업보고서 약 5번(연결 재무제표). 연결이 없으면 별도(OFS)로 다시 요청한다.
2,400 종목이면 하루에 다 못 끝날 수 있다. DAILY_LIMIT 에 닿으면 멈추고, 내일 같은 셀을 다시 실행하면 이어받는다.
같은 연도가 여러 보고서에 나오면 가장 최근 보고서의 값을 쓴다 (정정/재작성 반영).
"""
def collect_dart(ticker, corp_code, acc_mt):
    """한 종목의 연간 재무를 모아 (결산월, 데이터프레임) 으로 돌려준다."""
    if not acc_mt:
        js = dart_json('company', corp_code=corp_code)
        if js is None:
            raise NoData(ticker)
        acc_mt = str(js.get('acc_mt') or '').strip()
    if acc_mt != '12':
        raise NonDecember(ticker)

    parts = []
    for yr in request_years():
        js = dart_json('fnlttSinglAcntAll', corp_code=corp_code, bsns_year=str(yr),
                       reprt_code='11011', fs_div='CFS')
        if js is None:                                          # 연결 재무제표가 없으면 별도로
            js = dart_json('fnlttSinglAcntAll', corp_code=corp_code, bsns_year=str(yr),
                           reprt_code='11011', fs_div='OFS')
        if js is None:
            continue
        d = parse_report(js['list'], yr)
        d['rpt_year'] = yr
        parts.append(d)

    if not parts:
        raise NoData(ticker)
    df = pd.concat(parts).sort_values('rpt_year')
    df = df.drop_duplicates(['계정', '기준일'], keep='last')        # 최근 보고서 값이 남는다
    return acc_mt, df[['계정', '기준일', '값']]


engine, con = db_connect()
mycursor = con.cursor()

codes = set(pd.read_sql("""
select 종목코드 from kor_ticker
where 기준일 = (select max(기준일) from kor_ticker) and 종목구분 = '보통주';
""", con=engine)['종목코드'])
```

```
if 'kor_delisted' in inspect(engine).get_table_names():
    codes |= set(pd.read_sql('select 종목코드 from kor_delisted;', con=engine)['종목코드'])

corp_map = pd.read_sql('select corp_code, 종목코드, acc_mt from dart_corp;', con=engine)
corp_map = corp_map.dropna(subset=['종목코드']).drop_duplicates('종목코드').set_index('종목코드')

since = (date.today() - timedelta(days=DART_REFRESH_DAYS)).strftime('%Y-%m-%d')
done = set(pd.read_sql(f"""
select 종목코드 from crawl_log where 대상 = 'dart' and 완료일 >= '{since}';
""", con=engine)['종목코드'])

no_corp = sorted(c for c in codes if c not in corp_map.index)
todo = [c for c in sorted(codes) if c in corp_map.index and c not in done]
print(f'대상 {len(codes)}개 / 고유번호 없음 {len(no_corp)}개 / 최근 처리 완료 {len(done)}개 -> 이번에 받을 종목 {len(todo)}
개')

query = """
    insert into dart_fs (종목코드, 계정, 기준일, 값)
    values (%s,%s,%s,%s) as new
    on duplicate key update 값 = new.값
"""

error_list = []
br = Breaker()

try:
    for ticker in tqdm(todo):
        corp_code = corp_map.loc[ticker, 'corp_code']
        acc_mt = corp_map.loc[ticker, 'acc_mt']
        acc_mt = acc_mt if isinstance(acc_mt, str) else ''
        try:
            acc_mt, df = collect_dart(ticker, corp_code, acc_mt)
            df['종목코드'] = ticker
            mycursor.executemany(query, df[['종목코드', '계정', '기준일', '값']].values.tolist())
            mycursor.execute('update dart_corp set acc_mt = %s where corp_code = %s', (acc_mt, corp_code))
            con.commit()
            log_done(mycursor, con, 'dart', ticker, 'ok')
            br.ok()

        except (DartLimit, DartFatal):
            raise
        except NonDecember:
            mycursor.execute('update dart_corp set acc_mt = %s where corp_code = %s', ('xx', corp_code))
            con.commit()
            log_done(mycursor, con, 'dart', ticker, 'nondec')
        except NoData:
            log_done(mycursor, con, 'dart', ticker, 'nodata')
        except Exception as e:
            print(ticker, type(e).__name__, str(e)[:80])
            error_list.append(ticker)
            br.fail()

except (DartLimit, DartFatal, BlockedError) as e:
    print('중단:', e)

finally:
    engine.dispose()
    con.close()

print('실패 종목 수:', len(error_list), '/ 오늘 보낸 호출 수:', n_calls)


#%% [DART-5] 수집 결과 + 계정 매핑 커버리지 점검
engine, con = db_connect()

cov = pd.read_sql("""
select 계정, count(distinct 종목코드) as 종목수, min(기준일) as 시작, max(기준일) as 끝, count(*) as 행수
from dart_fs group by 계정 order by 종목수 desc;
""", con=engine)
print(cov.to_string(index=False))

n_tk = pd.read_sql('select count(distinct 종목코드) as n from dart_fs;', con=engine)['n'][0]
```

```
print('\n재무가 들어온 종목 수:', n_tk)
print('계정별 종목 수가 종목 수보다 많이 모자라면 해당 계정의 MAP 후보(account_id/계정명)가 맞지 않는 것입니다.')

yrs = pd.read_sql("""
select 종목코드, count(*) as 연수 from dart_fs where 계정 = 'ni' group by 종목코드;
""", con=engine)
print('\n순이익 기준 종목별 연간 재무 연수 분포')
print(yrs['연수'].value_counts().sort_index().to_string())

print(pd.read_sql("""
select 상태, count(*) as 종목수 from crawl_log where 대상 = 'dart' group by 상태;
""", con=engine))

engine.dispose()
con.close()


def show_accounts(ticker, bsns_year):
    """MAP 이 안 맞는 회사의 실제 계정 구성을 보고 싶을 때 쓴다. (호출 1~2건)"""
    engine, con = db_connect()
    cc = pd.read_sql('select corp_code from dart_corp where 종목코드 = %(t)s;',
                     con=engine, params={'t': ticker})['corp_code'][0]
    engine.dispose()
    con.close()
    js = dart_json('fnlttSinglAcntAll', corp_code=cc, bsns_year=str(bsns_year),
                   reprt_code='11011', fs_div='CFS')
    if js is None:
        print('데이터 없음')
        return None
    d = pd.DataFrame(js['list'])[['sj_div', 'account_id', 'account_nm', 'thstrm_amount']]
    print(d.to_string())
    return d
# 사용 예:  show_accounts('005930', 2023)


#%% [DART-6] (별도) 주가 과거 보강
"""
재무를 2013년부터 받아도 kor_price 가 5년치면 백테스트 기간이 5년에 묶인다.
DB 의 첫 날짜가 PRICE_FROM 보다 늦은 종목은 PRICE_FROM 부터 오늘까지 전체를 다시 받는다.
(일부만 이어 붙이면 그 사이에 액면분할 등이 있을 때 수정주가가 어긋나기 때문이다)
네이버 차트가 얼마나 먼 과거까지 주는지는 확인 안 됨. 먼저 한 종목으로 확인한다.
"""
PRICE_FROM = '2015-01-01'
SLEEP_PRICE = (0.3, 0.8)

test = None
try:
    r = safe_call(lambda: rq.get(
        'https://fchart.stock.naver.com/siseJson.nhn?symbol=005930&requestType=1'
        f'&startTime={PRICE_FROM.replace("-", "")}&endTime={date.today():%Y%m%d}&timeframe=day', timeout=20))
    test = pd.read_csv(BytesIO(r.content))
    first = re.findall(r'\d+', str(test.iloc[0, 0]))[0]
    print('삼성전자 테스트: 받은 첫 날짜', first, '/ 행 수', len(test))
except Exception as e:
    print('테스트 실패:', type(e).__name__, str(e)[:100])


def fetch_price(ticker, fr, to):
    """네이버 차트에서 일봉을 받아 정리한다. 결과 열: 날짜, 시가, 고가, 저가, 종가, 거래량, 종목코드"""
    url = (f'https://fchart.stock.naver.com/siseJson.nhn?symbol={ticker}&requestType=1'
           f'&startTime={fr}&endTime={to}&timeframe=day')

    def _get():
        r = rq.get(url, timeout=20)
        r.raise_for_status()
        return r.content

    data_price = pd.read_csv(BytesIO(safe_call(_get)))
    price = data_price.iloc[:, 0:6].copy()
    price.columns = ['날짜', '시가', '고가', '저가', '종가', '거래량']
    price = price.dropna()
    price['날짜'] = price['날짜'].astype(str).str.extract(r'(\d+)')
```

```
    price['날짜'] = pd.to_datetime(price['날짜'])
    price['종목코드'] = ticker
    if len(price) == 0:
        raise NoData(ticker)
    return price


engine, con = db_connect()
mycursor = con.cursor()

first_map = pd.read_sql('select 종목코드, min(날짜) as 첫날 from kor_price group by 종목코드;', con=engine)
first_map['첫날'] = pd.to_datetime(first_map['첫날'])
first_map = first_map.set_index('종목코드')['첫날']

since = (date.today() - timedelta(days=365)).strftime('%Y-%m-%d')
done_back = set(pd.read_sql(f"""
select 종목코드 from crawl_log where 대상 = 'pback' and 완료일 >= '{since}';
""", con=engine)['종목코드'])

limit = pd.Timestamp(PRICE_FROM) + pd.Timedelta(days=40)
todo = [t for t, d in first_map.items() if d > limit and t not in done_back]
print(f'과거 보강 대상 {len(todo)}개 (DB 첫 날짜가 {limit.date()} 보다 늦은 종목)')

query_price = """
    insert into kor_price(날짜, 시가, 고가, 저가, 종가, 거래량, 종목코드)
    values(%s,%s,%s,%s,%s,%s,%s) as new
    on duplicate key update
    시가 = new.시가, 고가 = new.고가, 저가 = new.저가, 종가 = new.종가, 거래량 = new.거래량;
"""

to = date.today().strftime('%Y%m%d')
fr = PRICE_FROM.replace('-', '')
error_list = []
br = Breaker()

try:
    for ticker in tqdm(todo):
        try:
            price = fetch_price(ticker, fr, to)
            mycursor.executemany(query_price, price.values.tolist())
            con.commit()
            log_done(mycursor, con, 'pback', ticker, 'ok')
            br.ok()
        except BlockedError:
            raise
        except NoData:
            log_done(mycursor, con, 'pback', ticker, 'nodata')
        except Exception as e:
            print(ticker, type(e).__name__, str(e)[:80])
            error_list.append(ticker)
            br.fail()
        nap(SLEEP_PRICE)
except BlockedError as e:
    print('중단:', e)
finally:
    engine.dispose()
    con.close()

print('실패 종목 수:', len(error_list))
```

부록 C. 4_상장폐지_수집.py

```
#%% [상폐-0] 안내
"""
상장폐지 종목 수집 (생존편향 완화)   파일명: 4_상장폐지_수집.py

[왜 필요한가]
kor_ticker 는 지금 상장된 종목뿐이라, 백테스트 기간 중에 상장폐지된 종목이 빠져 있다.
이러면 망한 종목이 사라져 수익률이 실제보다 좋게 나온다(생존편향).

[이 파일이 하는 일]
FinanceDataReader 의 상장폐지 종목 목록과 가격을 받아 DB 에 넣는다.
  kor_delisted        : 종목코드, 종목명, 상장폐지일, 사유
  kor_price_delisted  : 날짜, 종목코드, 종가, 시가총액

[실행 순서]
[상폐-1] 설정 -> [상폐-2] 점검 -> [상폐-3] 목록 저장 -> [상폐-4] 가격 수집 -> [상폐-5] 확인
그리고 3_장기데이터_수집.py 를 실행하면 폐지 종목의 재무도 같이 받는다.

[패키지]
  conda activate ta_env
  python -m pip install finance-datareader

[출처와 확인 안 된 것]
호출 방식은 공식 문서가 아니라 제3자 블로그(algolab, 2026-09-03)와 FinanceDataReader 소스에서 가져왔다.
  fdr.StockListing('KRX-DELISTING')                    상장폐지 목록 (열: Symbol Name Market SecuGroup ListingDate
DelistingDate Reason ...)
  fdr.DataReader('KRX-DELISTING:종목코드', 시작, 끝)       가격 (열: Open High Low Close Volume Amount MarCap ...)
- 가격 호출은 KRX 에 로그인 없이 요청한다. 로그인이 필요해져서 400 오류가 나면 아래 get_delist_price 가
  1번 파일처럼 pykrx 로그인 세션(KRX_ID / KRX_PW)으로 같은 요청을 보내는 방식으로 넘어간다. (KRX 가 실제로 어떤 방식을 받는지는 확인
안 됨)
- MarCap 의 단위(원인지), 종가가 수정주가인지는 확인 안 됐다. [상폐-2] 에서 MarCap / 종가 를 목록의 ListingShares 와 비교해 본다.
- 폐지 종목 종가가 수정주가가 아니면 액면분할이 있었던 종목의 수익률이 틀어진다.
- 목록에는 주식이 아닌 것(펀드, ELW 등)과 1960년대 폐지 종목까지 4천 개 넘게 들어 있어서, SecuGroup 이 '주권'인 것만 쓴다.
"""


#%% [상폐-1] 설정 + 공통 도구 + 테이블 준비
import os
import re
import time
import random
import getpass
from datetime import date, timedelta
from urllib.parse import quote_plus

import numpy as np
import pandas as pd
import pymysql
from sqlalchemy import create_engine
from tqdm import tqdm
import FinanceDataReader as fdr

PRICE_FROM = '2015-01-01'     # 이 날짜 이후에 상장폐지된 종목만 대상으로 한다
SLEEP_DL = (0.5, 1.2)
MAX_CONSEC_FAIL = 10

# 목록의 열 이름을 자동으로 찾지 못하면 [상폐-2] 출력을 보고 여기에 직접 적는다
CODE_COL = None
NAME_COL = None
DATE_COL = None
REASON_COL = None

DB_USER = 'root'
DB_HOST = '127.0.0.1'
DB_NAME = 'stock_db'
DB_PW = os.environ.get('DB_PW') or getpass.getpass('DB password: ')


def db_connect():
    engine = create_engine(
        f'mysql+pymysql://{DB_USER}:{quote_plus(DB_PW)}@{DB_HOST}:3306/{DB_NAME}')
```

```
    con = pymysql.connect(user=DB_USER, password=DB_PW, host=DB_HOST, database=DB_NAME, charset='utf8')
    return engine, con


class BlockedError(Exception):
    pass


class NoData(Exception):
    pass


class Breaker:
    def __init__(self, limit=MAX_CONSEC_FAIL):
        self.n = 0
        self.limit = limit

    def ok(self):
        self.n = 0

    def fail(self):
        self.n += 1
        if self.n >= self.limit:
            raise BlockedError(f'연속 {self.n}번 실패 -> 중단합니다. 시간을 두고 다시 실행하면 이어서 받습니다.')


def nap(rng):
    time.sleep(random.uniform(*rng))


def safe_call(fn, tries=3, base=3):
    for k in range(tries):
        try:
            return fn()
        except Exception:
            if k == tries - 1:
                raise
            time.sleep(base * (2 ** k) + random.uniform(0, 1))


def log_done(cur, con, kind, ticker, status):
    cur.execute("""
        insert into crawl_log (대상, 종목코드, 완료일, 상태)
        values (%s,%s,%s,%s) as new
        on duplicate key update 완료일 = new.완료일, 상태 = new.상태
    """, (kind, ticker, date.today().strftime('%Y-%m-%d'), status))
    con.commit()


# ---- KRX 가격 호출 (FinanceDataReader 먼저, 400 등으로 실패하면 로그인 세션 방식으로 전환) ----
KRX_URL = 'https://data.krx.co.kr/comm/bldAttendant/getJsonData.cmd'
KRX_HEADERS = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/120.0.0.0 Safari/537.36',
    'Referer': 'https://data.krx.co.kr/contents/MDC/MDI/outerLoader/index.cmd',
}
DL_METHOD = 'auto'            # 'auto': fdr 먼저 / 'fdr': fdr 만 / 'session': 로그인 세션만
_krx_sess = None
_full_code = None


def krx_session():
    """1번 파일과 같은 방식의 KRX 로그인 세션 (환경변수 KRX_ID, KRX_PW 사용)."""
    global _krx_sess
    if _krx_sess is None:
        from pykrx.website.comm.auth import get_auth_session
        _krx_sess = get_auth_session()
    return _krx_sess


def krx_post(data):
    r = krx_session().post(KRX_URL, data=data, headers=KRX_HEADERS, timeout=30)
```

```
    r.raise_for_status()
    if r.text.lstrip()[:1] == '<' or 'LOGOUT' in r.text[:50]:
        raise BlockedError('KRX 가 JSON 대신 HTML 을 보냈습니다 (로그인 풀림 또는 차단 의심)')
    return r.json()


def get_full_code(short_code):
    """상장폐지 종목의 짧은 코드 -> KRX 내부 전체 코드 (FinanceDataReader 소스와 같은 조회)."""
    global _full_code
    if _full_code is None:
        j = krx_post({'mktsel': 'ALL', 'searchText': '', 'bld': 'dbms/comm/finder/finder_listdelisu'})
        _full_code = {b['short_code']: b['full_code'] for b in j['block1']}
    return _full_code.get(short_code)


def to_n(sr):
    return pd.to_numeric(sr.astype(str).str.replace(',', ''), errors='coerce')


def delist_price_session(ticker, start, end):
    """KRX [MDCSTAT23902] 를 로그인 세션으로 2년씩 나눠 요청한다. 결과: Date 인덱스, Close, MarCap 열."""
    full = get_full_code(ticker)
    if full is None:
        raise NoData(ticker)
    s = pd.Timestamp(start)
    e_all = pd.Timestamp(end)
    parts = []
    while True:
        e = min(s + pd.DateOffset(years=2) - pd.Timedelta(days=1), e_all)
        j = krx_post({'bld': 'dbms/MDC/STAT/issue/MDCSTAT23902', 'isuCd': full, 'isuCd2': '',
                      'strtDd': s.strftime('%Y%m%d'), 'endDd': e.strftime('%Y%m%d'),
                      'share': '1', 'money': '1', 'csvxls_isNo': 'false'})
        out = j.get('output') or []
        if out:
            parts.append(pd.DataFrame(out))
        if e >= e_all:
            break
        s = e + pd.Timedelta(days=1)
        nap((0.3, 0.7))
    if not parts:
        return pd.DataFrame()
    df = pd.concat(parts)
    res = pd.DataFrame({'Date': pd.to_datetime(df['TRD_DD']),
                        'Close': to_n(df['TDD_CLSPRC']),
                        'MarCap': to_n(df['MKTCAP'])})
    return res.set_index('Date').sort_index()


def get_delist_price(ticker, start, end):
    global DL_METHOD
    if DL_METHOD in ('auto', 'fdr'):
        try:
            p = fdr.DataReader(f'KRX-DELISTING:{ticker}', start, end)
            if DL_METHOD == 'auto':
                DL_METHOD = 'fdr'
            return p
        except Exception as e:
            if DL_METHOD == 'fdr':
                raise
            print('FinanceDataReader 호출 실패 -> 로그인 세션 방식으로 전환:', type(e).__name__, str(e)[:80])
            DL_METHOD = 'session'
    return delist_price_session(ticker, start, end)


DDL = [
"""
create table if not exists kor_delisted
(
    종목코드 varchar(6) not null,
    종목명 varchar(50),
    상장폐지일 date,
    사유 varchar(100),
```

```
    primary key(종목코드)
);
""",
"""
create table if not exists kor_price_delisted
(
    날짜 date,
    종목코드 varchar(6),
    종가 double,
    시가총액 double,
    primary key(날짜, 종목코드)
);
""",
"""
create table if not exists crawl_log
(
    대상 varchar(10),
    종목코드 varchar(6),
    완료일 date,
    상태 varchar(10),
    primary key(대상, 종목코드)
);
""",
]

engine, con = db_connect()
mycursor = con.cursor()
for q in DDL:
    mycursor.execute(q)
con.commit()
engine.dispose()
con.close()
print('테이블 준비 완료')


#%% [상폐-2] 점검: 상장폐지 목록과 가격이 실제로 오는지 확인
"""
여기서 멈추고 출력을 먼저 확인한다.
- 목록은 4천 개가 넘고 펀드, 1960년대 폐지 종목, 8자리 코드(주식이 아닌 것)까지 섞여 있다. 그래서
  SecuGroup 이 '주권', 코드 6자리, 폐지일이 PRICE_FROM 이후인 것만 골라 그중 하나로 가격을 시험한다.
- 가격은 두 방식을 모두 시험한다: (1) FinanceDataReader  (2) KRX 로그인 세션
- 시가총액 / 종가 가 목록의 ListingShares(상장주식수) 와 비슷하면 MarCap 은 원 단위, 종가는 그 시점 주가다.
"""
dl_raw = fdr.StockListing('KRX-DELISTING')
print('상장폐지 목록 행 수:', len(dl_raw))
print('열 이름:', list(dl_raw.columns))


def find_col(df, keys, manual=None):
    if manual:
        return manual
    for c in df.columns:
        lc = str(c).lower()
        if any(k in lc for k in keys):
            return c
    return None


code_col = find_col(dl_raw, ['symbol', 'code', '종목코드'], CODE_COL)
name_col = find_col(dl_raw, ['name', '종목명'], NAME_COL)
date_col = find_col(dl_raw, ['delisting', '폐지일'], DATE_COL)
reason_col = find_col(dl_raw, ['reason', '사유'], REASON_COL)
secu_col = find_col(dl_raw, ['secugroup', '증권구분'])
shares_col = find_col(dl_raw, ['listingshares', '상장주식수'])
print('\n자동 탐지:', dict(code=code_col, name=name_col, date=date_col, reason=reason_col,
                          secu=secu_col, shares=shares_col))
if code_col is None or date_col is None:
    raise RuntimeError('종목코드/폐지일 열을 못 찾았습니다. 위 열 이름을 보고 CODE_COL, DATE_COL 을 적어 주세요.')

if secu_col:
    print('\n증권구분(SecuGroup) 종류')
    print(dl_raw[secu_col].value_counts().to_string())
```

```
cand = dl_raw.copy()
cand['_code'] = cand[code_col].astype(str).str.strip()
cand['_dt'] = pd.to_datetime(cand[date_col], errors='coerce')
mask = (cand['_code'].str.fullmatch(r'[0-9A-Z]{6}')
        & (cand['_dt'] >= pd.Timestamp(PRICE_FROM))
        & (cand['_code'].str[-1:] == '0'))
if secu_col:
    mask &= cand[secu_col].astype(str).str.contains('주권')
cand = cand[mask].sort_values('_dt', ascending=False).reset_index(drop=True)
print(f'\n조건(주권, 6자리 코드, 보통주, 폐지일 >= {PRICE_FROM}) 통과: {len(cand)}개')
show = [c for c in [code_col, name_col, 'Market', secu_col, date_col, reason_col, shares_col] if c]
print(cand[show].head(8).to_string())
if len(cand) == 0:
    raise RuntimeError('조건을 통과한 종목이 없습니다. 위 열 이름과 SecuGroup 종류를 붙여 주세요.')

# 폐지된 지 1~2년 된 종목 하나로 시험한다 (너무 오래되면 데이터가 없을 수 있다)
row = cand.iloc[min(len(cand) - 1, 20)]
sample = row['_code']
sample_end = row['_dt'].strftime('%Y-%m-%d')
print('\n점검 종목:', sample, row[name_col], '/ 폐지일', sample_end)


def show_price(tag, p):
    print(f'\n[{tag}] 행 수: {len(p)} / 열: {list(p.columns)}')
    if len(p) == 0:
        return
    print(p[[c for c in ['Close', 'MarCap'] if c in p.columns]].tail(5).to_string())
    if 'MarCap' in p.columns and 'Close' in p.columns:
        implied = (p['MarCap'] / p['Close']).iloc[-1]
        print('마지막 날 MarCap / Close =', f'{implied:,.0f}',
              '/ 목록의 상장주식수 =', row[shares_col] if shares_col else '확인 불가')


try:
    show_price('1) FinanceDataReader', fdr.DataReader(f'KRX-DELISTING:{sample}', PRICE_FROM, sample_end))
except Exception as e:
    print('\n[1) FinanceDataReader] 실패:', type(e).__name__, str(e)[:150])

try:
    show_price('2) KRX 로그인 세션', delist_price_session(sample, PRICE_FROM, sample_end))
except Exception as e:
    print('\n[2) KRX 로그인 세션] 실패:', type(e).__name__, str(e)[:150])


#%% [상폐-3] 목록 정리 + kor_delisted 저장
if secu_col:
    dl_raw = dl_raw[dl_raw[secu_col].astype(str).str.contains('주권')]         # 주식(주권)만. 펀드, ELW 등 제외

dl = pd.DataFrame({
    '종목코드': dl_raw[code_col].astype(str).str.strip(),
    '종목명': dl_raw[name_col].astype(str).str.strip() if name_col else '',
    '상장폐지일': pd.to_datetime(dl_raw[date_col], errors='coerce'),
    '사유': dl_raw[reason_col].astype(str).str.strip() if reason_col else '',
})

n0 = len(dl)
dl = dl.dropna(subset=['상장폐지일'])
dl = dl[dl['상장폐지일'] >= pd.Timestamp(PRICE_FROM)]                 # 기간 안에 폐지된 종목만
dl = dl[dl['종목코드'].str.fullmatch(r'[0-9A-Z]{6}')]                   # 6자리 코드만 (8자리는 주식이 아닌 상품)
dl = dl[dl['종목코드'].str[-1:] == '0']                                 # 우선주 제외 (끝자리 0 이 아님)
dl = dl[~dl['종목명'].str.contains('스팩|제[0-9]+호')]                  # 스팩 제외
dl = dl[~dl['종목명'].str.endswith('리츠')]                             # 리츠 제외
dl = dl.drop_duplicates('종목코드', keep='last').reset_index(drop=True)
print(f'목록 {n0}개 -> 조건 통과 {len(dl)}개')
print('폐지 연도별 종목 수')
print(dl['상장폐지일'].dt.year.value_counts().sort_index().to_string())
if reason_col:
    print('\n폐지 사유 상위')
    print(dl['사유'].value_counts().head(10).to_string())

engine, con = db_connect()
```

```
mycursor = con.cursor()
query = """
    insert into kor_delisted (종목코드, 종목명, 상장폐지일, 사유)
    values (%s,%s,%s,%s) as new
    on duplicate key update 종목명 = new.종목명, 상장폐지일 = new.상장폐지일, 사유 = new.사유
"""
args = dl[['종목코드', '종목명', '상장폐지일', '사유']].copy()
args['상장폐지일'] = args['상장폐지일'].dt.strftime('%Y-%m-%d')
args['사유'] = args['사유'].str.slice(0, 100)
mycursor.executemany(query, args.values.tolist())
con.commit()
engine.dispose()
con.close()
print('kor_delisted 저장 완료:', len(dl))


#%% [상폐-4] 가격 수집 (이어받기)
"""
폐지일 이전 가격을 PRICE_FROM 부터 받는다. 오늘 처리한 종목은 crawl_log 를 보고 건너뛴다.
FinanceDataReader 가 내부에서 KRX 에 여러 번 요청할 수 있어서 종목 사이에 쉬고, 연속 실패하면 멈춘다.
"""
def pick_col(df, names):
    low = {str(c).lower(): c for c in df.columns}
    for n in names:
        if n.lower() in low:
            return low[n.lower()]
    return None


engine, con = db_connect()
mycursor = con.cursor()

targets = pd.read_sql('select 종목코드, 상장폐지일 from kor_delisted;', con=engine)
today_str = date.today().strftime('%Y-%m-%d')
done = set(pd.read_sql(f"""
select 종목코드 from crawl_log where 대상 = 'dprice' and 완료일 >= '{(date.today() -
timedelta(days=180)).strftime('%Y-%m-%d')}';
""", con=engine)['종목코드'])
todo = targets[~targets['종목코드'].isin(done)].reset_index(drop=True)
print(f'대상 {len(targets)}개 중 최근 처리 {len(done)}개 -> 이번에 받을 종목 {len(todo)}개')

query_price = """
    insert into kor_price_delisted (날짜, 종목코드, 종가, 시가총액)
    values (%s,%s,%s,%s) as new
    on duplicate key update 종가 = new.종가, 시가총액 = new.시가총액
"""

error_list = []
br = Breaker()

try:
    for i in tqdm(range(len(todo))):
        ticker = todo['종목코드'][i]
        end = pd.to_datetime(todo['상장폐지일'][i]).strftime('%Y-%m-%d')
        try:
            p = safe_call(lambda: get_delist_price(ticker, PRICE_FROM, end))
            if p is None or len(p) == 0:
                raise NoData(ticker)

            c_close = pick_col(p, ['Close'])
            c_mcap = pick_col(p, ['MarCap', 'Marcap'])
            if c_close is None:
                raise ValueError('Close 열이 없습니다: ' + str(list(p.columns)))

            out = pd.DataFrame({
                '날짜': pd.to_datetime(p.index).strftime('%Y-%m-%d'),
                '종목코드': ticker,
                '종가': p[c_close].values,
                '시가총액': p[c_mcap].values if c_mcap else np.nan,
            })
            out = out.replace([np.inf, -np.inf], np.nan)
            out = out.astype(object).where(out.notna(), None)
```

```
            mycursor.executemany(query_price, out[['날짜', '종목코드', '종가', '시가총액']].values.tolist())
            con.commit()
            log_done(mycursor, con, 'dprice', ticker, 'ok')
            br.ok()

        except BlockedError:
            raise
        except NoData:
            log_done(mycursor, con, 'dprice', ticker, 'nodata')
        except Exception as e:
            print(ticker, type(e).__name__, str(e)[:80])
            error_list.append(ticker)
            br.fail()

        nap(SLEEP_DL)

except BlockedError as e:
    print('중단:', e)

finally:
    engine.dispose()
    con.close()

print('실패 종목 수:', len(error_list))


#%% [상폐-5] 수집 결과 확인
engine, con = db_connect()

print(pd.read_sql("""
select 'kor_delisted' as 테이블, count(*) as 행수, max(상장폐지일) as 최근일 from kor_delisted
union all
select 'kor_price_delisted', count(*), max(날짜) from kor_price_delisted;
""", con=engine))

print(pd.read_sql("""
select 상태, count(*) as 종목수 from crawl_log where 대상 = 'dprice' group by 상태;
""", con=engine))

n_with_price = pd.read_sql('select count(distinct 종목코드) as n from kor_price_delisted;', con=engine)['n'][0]
print('가격이 들어온 폐지 종목 수:', n_with_price)

engine.dispose()
con.close()
```

부록 D. 2_그레이엄_분석백테스트.py

```
#%% [그레이엄-0] 안내
"""
벤저민 그레이엄 방어적 투자자 기준: 종목 분석 + 백테스트 (엄격 버전 / 현실 버전)

먼저 '1_자료수집.py' 를 보고 오세요.
이미 DB(stock_db)에 kor_ticker, kor_price, kor_fs 가 있으면 여기서부터 진행하세요.

[데이터 선택 - [그레이엄-1] 맨 위 두 줄]
FS_SOURCE = 'fnguide' : 1_자료수집.py 가 받은 kor_fs (연간 4~5년)
FS_SOURCE = 'dart'    : 3_장기데이터_수집.py 가 받은 dart_fs (2013년~, 엄격 버전용)
INCLUDE_DELISTED = True : 4_상장폐지_수집.py 가 받은 상장폐지 종목도 대상에 넣는다 (생존편향 완화)

[이 파일이 하는 일]
1) 그레이엄 7가지 기준으로 종목 스크리닝 (지금 시점)
2) 재무 조건은 통과했지만 가격이 아직 비싼 종목의 '매수 상한가' 계산 (언제 사야 하나)
3) 과거 백테스트 두 가지
   - 정기 방식 : 매년 4월 1일에 통과 종목을 동일가중으로 사서 1년 보유
   - 신호 방식 : 재무 조건 통과 종목을 지켜보다가 가격이 매수 상한가 아래로 내려온 날 매수
4) 결과의 한계 자동 점검 + 기준값을 조금 바꿨을 때도 결과가 유지되는지(민감도)

[엄격 버전 / 현실 버전]
엄격 : 원전에 가까운 기간(10년 흑자, 10년 성장). 연간 재무가 10년치 있어야 계산된다.
현실 : DB 에 있는 연간 재무 연수에 맞춘 버전(기본 4년).
DB 에 연간 재무가 몇 년치뿐이면 엄격 버전은 통과 종목이 0개로 나오는 게 정상이다.

[DB 비밀번호]
코드에 쓰지 않는다. 환경변수 DB_PW 에 넣거나 실행할 때 입력한다.

[패키지]
기존 + matplotlib.   conda activate ta_env  ->  python -m pip install matplotlib
"""


#%% [그레이엄-1] 설정 + DB 연결 + 테이블 확인
import os
import getpass
import runpy
from urllib.parse import quote_plus

import numpy as np
import pandas as pd
from sqlalchemy import create_engine, inspect

RUN_DOWNLOAD = False   # True: 자료수집 파일을 실행한다 / False: 건너뛰고 DB 에서 바로 읽는다
DOWNLOAD_FILE = r'C:\경로\1_자료수집.py'   # 본인 파일 위치로 고칠 것

FS_SOURCE = 'dart'           # 'fnguide': kor_fs (연간 4~5년) / 'dart': dart_fs (2013년~, 3_장기데이터_수집.py)
INCLUDE_DELISTED = True      # True: 상장폐지 종목 포함 (4_상장폐지_수집.py 를 먼저 실행해야 한다)

# True 로 하면 파일 전체가 실행되고 KRX 에도 접속한다. 차단이 걱정되면 False 로 둔다.
if RUN_DOWNLOAD:
    runpy.run_path(DOWNLOAD_FILE)

DB_USER = 'root'
DB_HOST = '127.0.0.1'
DB_NAME = 'stock_db'
DB_PW = os.environ.get('DB_PW') or getpass.getpass('DB password: ')

engine = create_engine(
    f'mysql+pymysql://{DB_USER}:{quote_plus(DB_PW)}@{DB_HOST}:3306/{DB_NAME}')

# 필요한 테이블이 없거나 비어 있으면 어떤 게 문제인지 알려주고 멈춘다
need_tables = ['kor_ticker', 'kor_price', 'kor_fs' if FS_SOURCE == 'fnguide' else 'dart_fs']
if INCLUDE_DELISTED:
    need_tables += ['kor_delisted', 'kor_price_delisted']
have_tables = inspect(engine).get_table_names()

missing = []
for t in need_tables:
    if t not in have_tables:
```

```
        missing.append(t)
    else:
        n = pd.read_sql(f'select count(*) as n from {t};', con=engine)['n'][0]
        if n == 0:
            missing.append(t + ' (비어 있음)')

if missing:
    raise RuntimeError('DB 에 없는 테이블: ' + ', '.join(missing) +
                       '\n-> RUN_DOWNLOAD = True 로 바꿔 다시 실행하거나, 1_자료수집.py(dart_fs 는 3번, 폐지 종목은 4번 파일)
를 먼저 실행하세요.')
print('테이블 확인 완료')


#%% [그레이엄-2] 계정명 확인
# 앞 이름은 이 코드에서 쓰는 짧은 이름, 뒤는 kor_fs 에 들어 있는 계정명이다.
# 확인된 건 매출액(수익), 당기순이익, 자본총계 뿐이고 유동자산/유동부채/비유동부채는 아직 확인 안 됨.
ACC = {
    'sales':  '매출액(수익)',
    'ni':     '당기순이익',
    'equity': '자본총계',
    'ca':     '유동자산',
    'cl':     '유동부채',
    'ltd':    '비유동부채',     # 장기부채 대용. 장기차입금+사채 계정이 있으면 그걸로 바꾸는 게 더 정확하다
}

if FS_SOURCE == 'fnguide':
    acc_in_db = pd.read_sql("""
    select distinct 계정 from kor_fs where 공시구분 = 'y';
    """, con=engine)['계정'].tolist()

    no_acc = [v for v in ACC.values() if v not in acc_in_db]
    if no_acc:
        print('kor_fs 에 없는 계정:', no_acc)
        print('비슷한 이름 후보:')
        print([a for a in acc_in_db if ('유동' in a) or ('부채' in a) or ('차입' in a)])
        raise RuntimeError('후보를 보고 ACC 의 계정명을 DB 에 있는 이름으로 바꾸세요.')
else:
    # dart_fs 는 3_장기데이터_수집.py 가 이미 짧은 이름(sales, ni ...)으로 저장해 둔다
    acc_in_db = pd.read_sql('select distinct 계정 from dart_fs;', con=engine)['계정'].tolist()
    no_acc = [k for k in ACC.keys() if k not in acc_in_db]
    if no_acc:
        print('dart_fs 에 없는 계정:', no_acc, '/ 있는 계정:', acc_in_db)
        raise RuntimeError('3_장기데이터_수집.py 의 [DART-5] 커버리지를 확인하고 MAP 을 고치세요.')
print('계정 확인 완료 (', FS_SOURCE, ')')


#%% [그레이엄-3] 기준값
"""
그레이엄 방어적 투자자 기준 (현명한 투자자). 기억 기준이라 원문과 한 번 더 대조해 볼 것.

1) 적정 규모      : 원전은 매출 1억 달러(제조업). 원화 환산은 가정이라 직접 정한 값이다.
2) 유동비율       : 유동자산 / 유동부채 가 2 이상
3) 장기부채       : 장기부채 <= 순유동자산 (유동자산 - 유동부채)
4) 이익 안정성    : 원전은 10년 연속 흑자
5) 이익 성장      : 10년간 주당이익 1/3 이상 증가 (처음 3년 평균 vs 마지막 3년 평균)
6) PER            : 최근 3년 평균이익 기준 15 이하
7) PBR            : 1.5 이하. 또는 PER x PBR 이 22.5 이하
                    PER 15 이하를 이미 요구하므로 실제로는 PER x PBR <= 22.5 가 PBR 조건을 포함한다
(배당 20년 연속은 DB 에 과거 배당이 없어서 뺐다)

[매수 상한가]
PER <= 15 와 PER x PBR <= 22.5 를 시가총액 기준으로 풀면
  시가총액 <= min( 15 x 3년 평균 순이익 ,  루트(22.5 x 3년 평균 순이익 x 자본총계) )
가 된다. 주식수가 약분되어 사라지므로 주식수 추정 오차의 영향을 받지 않는다.
이 값보다 시가총액이 낮아지는 날이 '진입 시점'이다.
"""
MIN_SALES = 1000           # 억원
MIN_CURRENT_RATIO = 2.0
EPS_GROWTH_MIN = 1 / 3
MAX_PER = 15
MAX_PER_PBR = 22.5         # 1.5 x 15
DISCLOSURE_LAG = 90        # 결산일 + 90일 이후에야 재무를 알 수 있다고 본다 (미래 정보 사용 방지)
```

```
REBAL_MD = '04-01'          # 매년 리밸런싱 날짜(월-일). 12월 결산 사업보고서가 나온 뒤
TRADING_COST = 0.003        # 한 번 사고팔 때 드는 비용 가정치(왕복)
TARGET_GAIN = 0.5           # 신호 방식 청산 기준. 매수가 대비 +50% 도달 시 매도
                            # (그레이엄이 저평가주에 제시한 규칙으로 기억하지만 원문 확인은 못 했다. 변수로 두고 바꿔 본다)

# 엄격 / 현실 버전에서 다른 값들
MODES = {
    'strict': dict(PROFIT_YEARS=10, GROWTH_WINDOW=10, GROWTH_AVG_YEARS=3, GROWTH_MIN_YEARS=10, PER_AVG_YEARS=3),
    'real':   dict(PROFIT_YEARS=4,  GROWTH_WINDOW=10, GROWTH_AVG_YEARS=1, GROWTH_MIN_YEARS=4,  PER_AVG_YEARS=3),
}
if FS_SOURCE == 'fnguide':
    # 확인된 사실: FnGuide 연간 재무는 결산 3개 연도(2023~2025)뿐이라, 현실 버전을 3년으로 줄여야 통과 종목이 나온다
    MODES['real'].update(PROFIT_YEARS=3, GROWTH_MIN_YEARS=3)
# PROFIT_YEARS    : 연속 흑자 연수
# GROWTH_WINDOW   : 이익 성장 비교 구간(연). 데이터가 짧으면 있는 만큼만 쓴다
# GROWTH_AVG_YEARS: 시작/끝 평균에 쓰는 연수 (원전 3)
# GROWTH_MIN_YEARS: 이익 성장 계산에 필요한 최소 연수
# PER_AVG_YEARS   : PER 에 쓰는 평균이익 연수


def set_mode(mode):
    """선택한 버전의 값을 전역 변수로 덮어쓴다. (스크리닝 함수가 전역 변수를 읽는다)"""
    globals().update(MODES[mode])


#%% [그레이엄-4] 데이터 불러오기
ticker_list = pd.read_sql("""
select 종목코드, 종목명, 시장구분, 종가, 시가총액, 기준일, 주당배당금
from kor_ticker
where 기준일 = (select max(기준일) from kor_ticker)
  and 종목구분 = '보통주';
""", con=engine)

if FS_SOURCE == 'fnguide':
    acc_sql = ', '.join(f"'{v}'" for v in ACC.values())
    fs_y = pd.read_sql(f"""
    select 종목코드, 계정, 기준일, 값
    from kor_fs
    where 공시구분 = 'y'
      and 계정 in ({acc_sql});
    """, con=engine)
    fs_y['계정'] = fs_y['계정'].map({v: k for k, v in ACC.items()})    # DB 계정명 -> 짧은 이름
else:
    keys_sql = ', '.join(f"'{k}'" for k in ACC.keys())
    fs_y = pd.read_sql(f"""
    select 종목코드, 계정, 기준일, 값
    from dart_fs
    where 계정 in ({keys_sql});
    """, con=engine)
fs_y['기준일'] = pd.to_datetime(fs_y['기준일'])

price = pd.read_sql("""
select 날짜, 종목코드, 종가 from kor_price;
""", con=engine)
price['날짜'] = pd.to_datetime(price['날짜'])
price_wide = price.pivot(index='날짜', columns='종목코드', values='종가').sort_index()
del price

# 과거 시가총액은 (현재 주식수 x 과거 수정주가) 로 근사한다.
# 수정주가는 액면분할 등이 반영되어 있어서 현재 주식수와 곱하면 대체로 맞지만,
# 유상증자나 자사주 소각이 많은 종목은 오차가 생긴다.
ticker_list['shares'] = (ticker_list['시가총액'] / ticker_list['종가']).replace([np.inf, -np.inf], np.nan)
shares = ticker_list.set_index('종목코드')['shares']
universe = set(ticker_list['종목코드'])
name_map = ticker_list.set_index('종목코드')[['종목명', '시장구분']]

# 시가총액(원) 표. 현재 상장 종목은 (수정주가 x 현재 주식수)
mcap_wide = price_wide.mul(shares.reindex(price_wide.columns), axis=1)

# 상장폐지 종목을 포함하면 가격표와 시가총액표에 폐지 종목 열을 붙인다 (시가총액은 FinanceDataReader 의 MarCap)
if INCLUDE_DELISTED:
```

```
    dl_info = pd.read_sql('select 종목코드, 종목명, 상장폐지일 from kor_delisted;', con=engine)
    dl_price = pd.read_sql('select 날짜, 종목코드, 종가, 시가총액 from kor_price_delisted;', con=engine)
    dl_price['날짜'] = pd.to_datetime(dl_price['날짜'])
    dl_price = dl_price[~dl_price['종목코드'].isin(price_wide.columns)]    # 겹치면 현재 상장 쪽 데이터를 쓴다
    dl_p = dl_price.pivot(index='날짜', columns='종목코드', values='종가')
    dl_m = dl_price.pivot(index='날짜', columns='종목코드', values='시가총액')
    all_idx = price_wide.index.union(dl_p.index)
    price_wide = price_wide.reindex(all_idx).join(dl_p)
    mcap_wide = mcap_wide.reindex(all_idx).join(dl_m)
    name_map = pd.concat([name_map, dl_info.set_index('종목코드')[['종목명']].assign(시장구분='폐지')])
    universe |= set(dl_p.columns)
    print('상장폐지 종목 포함:', dl_p.shape[1], '개 (시가총액이 있는 종목', int(dl_m.notna().any().sum()), '개)')

fs_y = fs_y[fs_y['종목코드'].isin(universe)]

# 데이터 범위 확인 (엄격/현실 버전 중 어느 쪽이 가능한지 여기서 판단한다)
n_years = fs_y[fs_y['계정'] == 'ni'].groupby('종목코드')['기준일'].nunique()
print('재무 출처:', FS_SOURCE, '/ 대상 종목 수:', len(universe))
print('재무 기준일 범위:', fs_y['기준일'].min().date(), '~', fs_y['기준일'].max().date())
print('주가 범위       :', price_wide.index.min().date(), '~', price_wide.index.max().date())
print('종목별 연간 재무 연수 분포 (연수: 종목 수)')
print(n_years.value_counts().sort_index())
print('-> 가장 많은 종목의 연수가', int(n_years.mode().iloc[0]), '년. '
      'PROFIT_YEARS(현실)가 이보다 크면 통과 종목이 없으니 3번 셀 MODES 의 real 값을 낮출 것')


#%% [그레이엄-5] 스크리닝 함수
FIN_COLS = ['sales_ok', 'cr_ok', 'ltd_ok', 'profit_ok', 'growth_ok']     # 가격과 무관한 재무 조건
PRICE_COLS = ['per_ok', 'pbr_ok']                                        # 가격에 따라 매일 바뀌는 조건
OK_COLS = FIN_COLS + PRICE_COLS


def calc_growth(s):
    """구간 안에서 처음 평균 대비 마지막 평균의 증가율. 이익이 음수/0 이면 계산하지 않는다."""
    s = s.dropna().values
    if len(s) < GROWTH_MIN_YEARS:
        return np.nan
    first = s[:GROWTH_AVG_YEARS].mean()
    last_ = s[-GROWTH_AVG_YEARS:].mean()
    return last_ / first - 1 if first > 0 else np.nan


def graham_screen(as_of):
    """as_of 시점에 알 수 있었던 정보만으로 7가지 기준을 평가한다."""
    as_of = pd.Timestamp(as_of)
    empty = pd.DataFrame({'pass_all': pd.Series(dtype=bool),
                          'pass_fin': pd.Series(dtype=bool),
                          'ceil_mcap': pd.Series(dtype=float)})

    # 1) 아직 공시되지 않았을 재무는 제외
    fs = fs_y[fs_y['기준일'] + pd.Timedelta(days=DISCLOSURE_LAG) <= as_of]
    if fs.empty:
        return empty

    wide = fs.pivot_table(index=['종목코드', '기준일'], columns='계정', values='값').reset_index()
    wide.columns.name = None
    wide = wide.sort_values(['종목코드', '기준일'])
    for c in ACC.keys():
        if c not in wide.columns:
            wide[c] = np.nan

    g = wide.groupby('종목코드')
    last = g.tail(1).set_index('종목코드')                      # 종목별 가장 최근 연간 재무

    # 2) 최근 PROFIT_YEARS 년이 모두 흑자인가 (데이터가 모자라면 탈락)
    profit_ok = (g.tail(PROFIT_YEARS).groupby('종목코드')['ni']
                 .agg(lambda s: s.count() == PROFIT_YEARS and bool((s > 0).all())))

    # 3) 이익 성장률 (주당이익 대신 당기순이익, 주식수 변화는 무시)
    growth = g.tail(GROWTH_WINDOW).groupby('종목코드')['ni'].agg(calc_growth)

    # 4) 최근 PER_AVG_YEARS 년 평균 당기순이익
```

```
    avg_ni = (g.tail(PER_AVG_YEARS).groupby('종목코드')['ni']
              .agg(lambda s: s.mean() if s.count() == PER_AVG_YEARS else np.nan))

    # 5) as_of 이전 마지막 거래일의 시가총액(억원). 상장폐지된 종목은 폐지 뒤에 값이 없어서 자동으로 빠진다
    mcap = mcap_wide.loc[:as_of].iloc[-1] / 100000000

    df = pd.DataFrame(index=last.index)
    df['sales'] = last['sales']
    df['current_ratio'] = last['ca'] / last['cl']
    df['nca'] = last['ca'] - last['cl']                          # 순유동자산
    df['ltd'] = last['ltd']
    df['growth'] = growth
    df['avg_ni'] = avg_ni
    df['mcap'] = mcap.reindex(df.index)
    df['PER3'] = df['mcap'] / df['avg_ni']
    df['PBR'] = df['mcap'] / last['equity']
    df['PER_PBR'] = df['PER3'] * df['PBR']

    # 매수 상한가(시가총액 기준). 이익이나 자본이 0 이하이면 계산하지 않는다
    pos = (df['avg_ni'] > 0) & (last['equity'] > 0)
    ceil = np.minimum(MAX_PER * df['avg_ni'],
                      np.sqrt(MAX_PER_PBR * df['avg_ni'].clip(lower=0) * last['equity'].clip(lower=0)))
    df['ceil_mcap'] = ceil.where(pos)
    df['gap'] = df['mcap'] / df['ceil_mcap'] - 1                 # 0 이하이면 상한가 아래(= 매수 가능)

    # NaN 이 들어간 비교는 False 가 되어 자동으로 탈락한다 (금융업은 유동자산 계정이 없어 여기서 빠진다)
    df['sales_ok'] = df['sales'] >= MIN_SALES
    df['cr_ok'] = df['current_ratio'] >= MIN_CURRENT_RATIO
    df['ltd_ok'] = df['ltd'] <= df['nca']
    df['profit_ok'] = profit_ok.reindex(df.index, fill_value=False).astype(bool)
    df['growth_ok'] = df['growth'] >= EPS_GROWTH_MIN
    df['per_ok'] = (df['PER3'] > 0) & (df['PER3'] <= MAX_PER)
    df['pbr_ok'] = (df['PBR'] > 0) & (df['PER_PBR'] <= MAX_PER_PBR)

    df['score'] = df[OK_COLS].sum(axis=1)
    df['pass_fin'] = df[FIN_COLS].all(axis=1)
    df['pass_all'] = df[OK_COLS].all(axis=1)

    df = df.join(name_map)
    return df[df.index.isin(universe)]


#%% [그레이엄-6] 지금 시점 스크리닝 + 진입 시점 후보
as_of_now = price_wide.index.max()
cols_show = ['종목명', '시장구분', 'sales', 'current_ratio', 'growth', 'PER3', 'PBR', 'PER_PBR', 'gap']

res_now = {}
for m in MODES:
    set_mode(m)
    r = graham_screen(as_of_now)
    res_now[m] = r
    print(f'===== {m} 버전 / 기준일 {as_of_now.date()} / 대상 {len(r)}개 =====')
    if len(r) == 0:
        print('재무 데이터가 없어 계산할 수 없습니다.')
        continue
    print('기준별 통과 종목 수')
    print(r[OK_COLS].sum().to_string())
    print('재무 조건 통과:', int(r['pass_fin'].sum()), '/ 모든 조건 통과:', int(r['pass_all'].sum()))

set_mode('real')
r = res_now['real']

# (1) 지금 사도 되는 종목 : 모든 조건 통과
print('\n[지금 매수 조건 충족] (현실 버전)')
print(r[r['pass_all']][cols_show].sort_values('PER_PBR').to_string())

# (2) 재무는 통과했지만 아직 비싼 종목 : gap 이 0 에 가까울수록 가격이 조금만 내려와도 진입 신호
watch = r[r['pass_fin'] & ~r['pass_all']].sort_values('gap')
print('\n[관심 목록: 재무 통과, 가격 대기] gap = 시가총액이 매수 상한가보다 몇 % 높은지')
print(watch[cols_show].head(30).to_string())

# (3) 참고 : 배당 (그레이엄은 20년 연속 배당을 요구하지만 DB 에는 현재 주당배당금만 있다)
```

```
res_div = r[r['pass_all']].join(ticker_list.set_index('종목코드')['주당배당금'])
print('\n[참고] 모든 조건 통과 종목 중 현재 배당이 있는 종목 수:', int((res_div['주당배당금'] > 0).sum()))


#%% [그레이엄-7] 백테스트 함수
"""
방식 A (정기): 매년 REBAL_MD 이후 첫 거래일에 스크리닝 -> 통과 종목 동일가중 매수 -> 다음 리밸런싱까지 보유.
방식 B (신호): 같은 날 '재무 조건만' 통과한 종목을 후보로 두고, 시가총액이 매수 상한가 아래로 내려온 첫 날 매수.
               매수가 대비 TARGET_GAIN 에 닿으면 매도, 아니면 다음 리밸런싱 날 정리한다.
               후보 한 종목에 배정한 돈은 매수 전까지 현금(수익률 0)으로 둔다.
비교 대상: 같은 방식으로 굴린 '전 종목 동일가중'.

한계 (결과를 볼 때 감안할 것)
- 생존편향 : kor_ticker 는 현재 상장 종목뿐이라 상장폐지 종목이 빠져 있다. 수익률이 실제보다 좋게 나올 수 있다.
- 시가총액 : 현재 주식수로 근사한 값
- 이익      : 주당이익이 아니라 당기순이익
- 기간      : 주가와 연간 재무가 동시에 있는 구간만 의미가 있고, 보통 표본이 몇 번 안 된다
- 거래 제약 : 거래정지, 호가 단위, 슬리피지는 반영하지 않았다
"""
idx = price_wide.index
bench_picks = [c for c in price_wide.columns if c in universe]


def make_rebal_dates():
    out = []
    for y in range(idx.min().year, idx.max().year + 1):
        t = pd.Timestamp(f'{y}-{REBAL_MD}')
        pos = idx.searchsorted(t)
        if t > idx.min() and pos < len(idx) - 20:      # 마지막 20거래일 안이면 보유 기간이 너무 짧아 제외
            out.append(idx[pos])
    return out


def run_backtest(dates, get_picks, cost=TRADING_COST):
    """방식 A. get_picks(날짜) -> 종목코드 리스트. 일별 가치(NAV)와 매수 종목 수 기록을 돌려준다."""
    value = 1.0
    navs = []
    log = []
    for k, d0 in enumerate(dates):
        d1 = dates[k + 1] if k + 1 < len(dates) else idx[-1]

        picks = [p for p in get_picks(d0)
                 if p in price_wide.columns and pd.notna(price_wide.at[d0, p])]

        if len(picks) == 0:
            seg_nav = pd.Series(value, index=price_wide.loc[d0:d1].index)      # 현금 보유
        else:
            value *= (1 - cost)                                                # 전량 교체 비용
            seg = price_wide.loc[d0:d1, picks].ffill()
            seg_nav = value * (seg / seg.iloc[0]).mean(axis=1)

        value = seg_nav.iloc[-1]
        navs.append(seg_nav if k == 0 else seg_nav.iloc[1:])                   # 경계일 중복 제거
        log.append((d0.date(), len(picks)))

    return pd.concat(navs), pd.DataFrame(log, columns=['리밸런싱일', '매수종목수'])


def run_signal_backtest(dates, cands, ceils, target=TARGET_GAIN, cost=TRADING_COST):
    """방식 B. cands[d]: 재무 조건 통과 종목, ceils[d][종목]: 그 시점에 계산한 매수 상한가(시가총액, 억원)."""
    value = 1.0
    navs = []
    log = []
    entries = []
    for k, d0 in enumerate(dates):
        d1 = dates[k + 1] if k + 1 < len(dates) else idx[-1]
        seg_all = price_wide.loc[d0:d1]
        mc_all = mcap_wide.loc[d0:d1]
        cl = [c for c in cands[d0] if c in price_wide.columns]

        slots = pd.DataFrame(1.0, index=seg_all.index, columns=cl)             # 종목별 배정금의 가치. 처음엔 현금 1.0
        n_entry = 0
```

```
        for c in cl:
            px = seg_all[c].ffill()
            mc = mc_all[c].ffill() / 100000000
            hit = ((mc <= ceils[d0].get(c, np.nan)) & px.notna()).values
            if not hit.any():
                continue

            i = int(hit.argmax())                                          # 상한가 아래로 처음 내려온 날 = 진입일
            rel = (px / px.iloc[i]).values.copy()
            rel[:i] = 1.0                                                  # 진입 전에는 현금

            up = np.where(rel[i:] >= 1 + target)[0]                        # 목표 수익 도달 시 매도, 이후 그 값 유지
            exit_date = seg_all.index[-1]
            exit_reason = '기간종료'
            if len(up):
                j = i + int(up[0])
                rel[j:] = rel[j]
                exit_date = seg_all.index[j]
                exit_reason = '목표도달'

            rel[i:] *= (1 - cost)
            slots[c] = rel
            n_entry += 1
            entries.append((d0.date(), c, seg_all.index[i].date(), float(px.iloc[i]),
                            exit_date.date(), exit_reason, float(rel[-1] - 1)))

        if len(cl) == 0:
            seg_nav = pd.Series(value, index=seg_all.index)
        else:
            seg_nav = value * slots.mean(axis=1)

        value = seg_nav.iloc[-1]
        navs.append(seg_nav if k == 0 else seg_nav.iloc[1:])
        log.append((d0.date(), len(cl), n_entry))

    entries = pd.DataFrame(entries, columns=['리밸런싱일', '종목코드', '진입일', '진입가',
                                             '청산일', '청산사유', '기간수익률'])
    entries = entries.merge(name_map['종목명'].reset_index(), on='종목코드', how='left')
    return (pd.concat(navs),
            pd.DataFrame(log, columns=['리밸런싱일', '후보종목수', '실제진입수']),
            entries)


def run_all(mode):
    """한 버전(strict / real)의 스크리닝 + 두 방식 백테스트 + 비교 대상. 통과 종목이 없으면 None."""
    set_mode(mode)
    dates = make_rebal_dates()

    scr = {d: graham_screen(d) for d in dates}
    picks = {d: list(s.index[s['pass_all']]) for d, s in scr.items()}
    cands = {d: list(s.index[s['pass_fin']]) for d, s in scr.items()}
    ceils = {d: s['ceil_mcap'].to_dict() for d, s in scr.items()}

    # 재무 데이터가 아직 없어서 후보가 없던 앞쪽 날짜는 버린다
    while dates and len(cands[dates[0]]) == 0:
        dates.pop(0)
    if not dates:
        return None

    nav_a, log_a = run_backtest(dates, lambda d: picks[d])
    nav_b, log_b, entries = run_signal_backtest(dates, cands, ceils)
    nav_m, _ = run_backtest(dates, lambda d: bench_picks)

    return dict(dates=dates, nav_a=nav_a, log_a=log_a, nav_b=nav_b, log_b=log_b,
                entries=entries, nav_m=nav_m)


#%% [그레이엄-8] 백테스트 실행 (엄격 / 현실)
results = {}
for m in MODES:
    out = run_all(m)
    if out is None:
```

```
        print(f'[{m}] 재무 조건을 통과하는 종목이 있는 리밸런싱 날짜가 없어 백테스트를 건너뜁니다. '
              '(필요한 재무 연수가 DB 보다 길거나 기준이 너무 엄격함)')
        continue
    results[m] = out
    print(f'\n[{m}] 리밸런싱 {len(out["dates"])}회')
    print(out['log_a'].merge(out['log_b'], on='리밸런싱일').to_string(index=False))

set_mode('real')
if 'real' in results:
    print('\n[현실 버전] 신호 방식 진입 내역 (언제 샀고 언제 팔았나)')
    print(results['real']['entries'].to_string(index=False))


#%% [그레이엄-9] 성과 비교
def perf(nav):
    ret = nav.pct_change().dropna()
    years = (nav.index[-1] - nav.index[0]).days / 365.25
    return pd.Series({
        '누적수익률': nav.iloc[-1] - 1,
        'CAGR': nav.iloc[-1] ** (1 / years) - 1 if years > 0 else np.nan,
        'MDD': (nav / nav.cummax() - 1).min(),
        '변동성(연)': ret.std() * np.sqrt(252),
        'Sharpe(무위험 0)': ret.mean() / ret.std() * np.sqrt(252) if ret.std() > 0 else np.nan,
        '기간(년)': years,
    })


import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False

if results:
    fig, axes = plt.subplots(1, len(results), figsize=(6 * len(results), 5), squeeze=False)
    for ax, (m, o) in zip(axes[0], results.items()):
        table = pd.concat([perf(o['nav_a']), perf(o['nav_b']), perf(o['nav_m'])], axis=1,
                          keys=['정기(A)', '신호(B)', '전종목 동일가중'])
        print(f'\n===== {m} 버전 =====')
        print(table.round(4).to_string())

        o['nav_a'].plot(ax=ax, label='정기(A)')
        o['nav_b'].plot(ax=ax, label='신호(B)')
        o['nav_m'].plot(ax=ax, label='전종목 동일가중')
        ax.set_title(f'그레이엄 {m} 버전')
        ax.legend()
    plt.tight_layout()
    plt.show()
else:
    print('백테스트 결과가 없습니다. [그레이엄-8] 의 메시지를 확인하세요.')


#%% [그레이엄-10] 결과의 한계 자동 점검
def yearly(nav):
    y = nav.groupby(nav.index.year).last()
    r = y.pct_change()
    r.iloc[0] = y.iloc[0] / nav.iloc[0] - 1
    return r


for m, o in results.items():
    print(f'\n===== {m} 버전 점검 =====')
    years = (o['nav_a'].index[-1] - o['nav_a'].index[0]).days / 365.25
    n_reb = len(o['dates'])
    avg_pick = o['log_a']['매수종목수'].mean()
    zero_pick = int((o['log_a']['매수종목수'] == 0).sum())
    n_entry = int(o['log_b']['실제진입수'].sum())

    print(f'기간 {years:.1f}년 / 리밸런싱 {n_reb}회 / 정기 방식 평균 매수종목 {avg_pick:.1f}개 / '
          f'현금 기간 {zero_pick}회 / 신호 방식 진입 {n_entry}건')

    warn = []
    if years < 5:
        warn.append('기간이 5년 미만이라 한두 해 시장 흐름이 결과를 좌우한다')
```

```
    if n_reb < 5:
        warn.append('리밸런싱이 5회 미만이라 통계적 결론을 내기 어렵다')
    if avg_pick < 10:
        warn.append('평균 매수 종목이 10개 미만이라 개별 종목 한두 개가 결과를 좌우한다')
    if n_entry < 20:
        warn.append('신호 방식 진입이 20건 미만이라 표본이 부족하다')
    if INCLUDE_DELISTED:
        warn.append('상장폐지 종목을 넣었다. MarCap 단위(원)는 확인했지만 폐지 종목 종가가 수정주가인지는 확인되지 않았다. '
                    '재무를 받지 못한 폐지 종목(감사의견 거절, 합병 등)은 빠져 있어 생존편향이 일부 남는다')
    else:
        warn.append('생존편향: 상장폐지 종목이 빠져 있어 수익률이 높게 나올 수 있다 (INCLUDE_DELISTED = True 로 완화)')
    if FS_SOURCE == 'fnguide':
        warn.append('재무가 4~5년치뿐이라 엄격 버전은 계산되지 않는다 (FS_SOURCE = "dart" 로 장기 재무를 쓴다)')
    warn.append('현재 상장 종목의 과거 시가총액은 현재 주식수 근사, 이익은 순이익(주당이익 아님)')
    warn.append('거래비용은 가정치, 슬리피지/거래정지는 미반영')
    for w in warn:
        print(' -', w)

    # 연도별 수익률: 초과수익이 한 해에만 몰려 있는지 본다
    yr = pd.concat([yearly(o['nav_a']), yearly(o['nav_b']), yearly(o['nav_m'])], axis=1,
                   keys=['정기(A)', '신호(B)', '전종목 동일가중'])
    yr['A-벤치'] = yr['정기(A)'] - yr['전종목 동일가중']
    print('\n연도별 수익률')
    print(yr.round(4).to_string())
    print('정기(A)가 벤치마크보다 나은 해:', int((yr['A-벤치'] > 0).sum()), '/', len(yr))

"""
[가능한 전략인지 보는 기준]
1) 초과수익이 여러 해에 나뉘어 있는가 (한 해에 몰려 있으면 우연일 가능성이 크다)
2) 매수 종목 수와 리밸런싱 횟수가 충분한가 (위 경고가 없는가)
3) 거래비용을 TRADING_COST 의 2~3배로 올려도 벤치마크보다 나은가
4) 기준값을 조금 바꿔도 결과가 비슷한가 -> 다음 셀(민감도)
이 중 하나라도 안 되면 '전략이 통한다'가 아니라 '이 표본에서는 그렇게 보인다' 정도로만 해석한다.
"""


#%% [그레이엄-11] 민감도: 기준값을 조금 바꿔도 결과가 유지되는가 (현실 버전)
base = dict(MAX_PER_PBR=MAX_PER_PBR, MIN_CURRENT_RATIO=MIN_CURRENT_RATIO)
rows = []
for pbr in [18, 22.5, 30]:
    for cr in [1.5, 2.0]:
        globals().update(MAX_PER_PBR=pbr, MIN_CURRENT_RATIO=cr)
        o = run_all('real')
        if o is None:
            rows.append((pbr, cr, np.nan, np.nan, np.nan, np.nan))
            continue
        pa, pb, pm = perf(o['nav_a']), perf(o['nav_b']), perf(o['nav_m'])
        rows.append((pbr, cr, pa['CAGR'], pb['CAGR'], pm['CAGR'], o['log_a']['매수종목수'].mean()))
globals().update(base)

sens = pd.DataFrame(rows, columns=['PER_PBR 상한', '유동비율 하한', 'CAGR 정기(A)', 'CAGR 신호(B)',
                                   'CAGR 벤치', 'A 평균종목수'])
print(sens.round(4).to_string(index=False))
# 기준값이 조금 바뀔 때 결과가 크게 흔들리면 그 전략은 신뢰하기 어렵다.


#%% [그레이엄-12] 구간별 성과: 특정 기간을 빼면 결과가 어떻게 달라지는가
# 2025년 이후 시장이 크게 올랐을 때, 그 구간을 빼도 같은 결론인지 본다.
# 일별 가치(NAV)를 구간으로 잘라 1.0 으로 다시 맞춘 뒤 성과를 계산한다. 벤치마크는 코스피 지수가 아니라 전종목 동일가중이다.
PERIODS = [
    ('전체',          None,         None),
    ('2025년 이전',   None,         '2024-12-31'),
    ('2025년 이후',   '2025-01-01', None),
]


def perf_range(nav, start=None, end=None):
    s = nav.loc[start:end]
    if len(s) < 60:
        return None
    s = s / s.iloc[0]
    ret = s.pct_change().dropna()
```

```
    years = (s.index[-1] - s.index[0]).days / 365.25
    return dict(CAGR=s.iloc[-1] ** (1 / years) - 1, 누적=s.iloc[-1] - 1, MDD=(s / s.cummax() - 1).min(),
                Sharpe=ret.mean() / ret.std() * np.sqrt(252) if ret.std() > 0 else np.nan, 기간=years)


for m, o in results.items():
    rows = []
    for label, a, b in PERIODS:
        pa, pb, pm = (perf_range(o[k], a, b) for k in ('nav_a', 'nav_b', 'nav_m'))
        if pa is None or pm is None:
            continue
        rows.append(dict(구간=label, 기간_년=round(pm['기간'], 1),
                         CAGR_A=pa['CAGR'], CAGR_B=pb['CAGR'], CAGR_벤치=pm['CAGR'],
                         초과_A=pa['CAGR'] - pm['CAGR'], 초과_B=pb['CAGR'] - pm['CAGR'],
                         MDD_A=pa['MDD'], MDD_벤치=pm['MDD'],
                         Sharpe_A=pa['Sharpe'], Sharpe_B=pb['Sharpe'], Sharpe_벤치=pm['Sharpe']))
    print(f'\n===== {m} 버전 구간별 성과 =====')
    print(pd.DataFrame(rows).round(4).to_string(index=False))
# 구간이 짧을수록 우연의 영향이 크다. 위 표는 '그 구간에서의 수치'이고 통계적 검정은 아니다.


#%% [그레이엄-13] 자본·순이익이 지배주주 기준인지 점검
# DART 의 자본총계/당기순이익은 비지배지분을 포함한 값이고, 시가총액은 지배주주 몫만 반영한다.
# 지주회사처럼 비지배지분이 큰 회사는 PBR 이 실제보다 낮게 나올 수 있어서 FnGuide 값과 대조한다.
acc_fn = pd.read_sql("select distinct 계정 from kor_fs where 공시구분 = 'y';", con=engine)['계정'].tolist()
print('kor_fs 의 지배/비지배 관련 계정:', [a for a in acc_fn if '지배' in a])

# 1) 같은 종목, 같은 기준일에서 DART 값과 FnGuide 값이 얼마나 다른가
chk = {'equity': '자본총계', 'ni': '당기순이익'}
fn = pd.read_sql("""
select 종목코드, 계정, 기준일, 값 from kor_fs
where 공시구분 = 'y' and 계정 in ('자본총계', '당기순이익');
""", con=engine)
fn['계정'] = fn['계정'].map({v: k for k, v in chk.items()})
fn['기준일'] = pd.to_datetime(fn['기준일'])

m = fs_y[fs_y['계정'].isin(chk.keys())].merge(
    fn, on=['종목코드', '계정', '기준일'], suffixes=('_dart', '_fn'))
m = m[m['값_fn'] != 0].copy()
m['ratio'] = m['값_dart'] / m['값_fn']
print('\n[DART / FnGuide 비율] 중앙값이 1 근처면 단위와 기준이 같다는 뜻')
for a in chk:
    x = m[m['계정'] == a]['ratio']
    if len(x) == 0:
        print(a, ': 겹치는 (종목, 기준일)이 없음 -> 기준일 형식 확인 필요')
        continue
    print(f'{a}: 겹친 행 {len(x)} / 중앙값 {x.median():.3f} / '
          f'0.97~1.03 안 {((x > 0.97) & (x < 1.03)).mean():.1%} / '
          f'1.03 초과(DART가 큼) {(x >= 1.03).mean():.1%} / 0.97 미만 {(x <= 0.97).mean():.1%}')

# 2) 지배주주 기준으로 다시 계산한 PBR, PER x PBR (FnGuide 에 해당 계정이 있을 때만)
# 지배주주 순이익 계정은 kor_fs 에 '(지배주주지분)당기순이익' 이라는 이름으로 있어서 아래 P_NI 와 이름이 다르다.
# 아래 이름이 DB 에 없으면 PER 은 연결 순이익 그대로 쓰고, 자본만 지배주주 기준으로 바꿔 PBR 과 PER x PBR 을 다시 계산한다.
P_EQ, P_NI = '지배주주지분', '지배주주순이익'
r = res_now['real']
ok = r[r['pass_all']].copy()

if P_EQ in acc_fn:
    pe = pd.read_sql(f"select 종목코드, 기준일, 값 from kor_fs where 공시구분 = 'y' and 계정 = '{P_EQ}';", con=engine)
    pe['기준일'] = pd.to_datetime(pe['기준일'])
    pe_last = pe.sort_values('기준일').groupby('종목코드').tail(1).set_index('종목코드')['값']
    ok['PBR_지배'] = ok['mcap'] / pe_last.reindex(ok.index)
    ok['PERPBR_지배'] = ok['PER3'] * ok['PBR_지배']

    if P_NI in acc_fn:
        pn = pd.read_sql(f"select 종목코드, 기준일, 값 from kor_fs where 공시구분 = 'y' and 계정 = '{P_NI}';",
con=engine)
        pn['기준일'] = pd.to_datetime(pn['기준일'])
        pn3 = pn.sort_values('기준일').groupby('종목코드').tail(3).groupby('종목코드')['값'].agg(
            lambda s: s.mean() if s.count() == 3 else np.nan)
        ok['PER3_지배'] = ok['mcap'] / pn3.reindex(ok.index)
        ok['PERPBR_지배'] = ok['PER3_지배'] * ok['PBR_지배']
```

```
        ok['여전히통과'] = (ok['PER3_지배'] > 0) & (ok['PER3_지배'] <= MAX_PER) & \
                           (ok['PBR_지배'] > 0) & (ok['PERPBR_지배'] <= MAX_PER_PBR)
    else:
        ok['여전히통과'] = (ok['PBR_지배'] > 0) & (ok['PERPBR_지배'] <= MAX_PER_PBR)

    print(f'\n현재 통과 {len(ok)}개 중 지배주주 기준으로도 통과: {int(ok["여전히통과"].sum())}개 '
          f'(FnGuide 값이 없는 종목은 탈락으로 센다)')
    show = ['종목명', 'PBR', 'PBR_지배', 'PER_PBR', 'PERPBR_지배', '여전히통과']
    ok['PBR차이'] = ok['PBR_지배'] / ok['PBR']
    print('\n[PBR 차이가 큰 종목 20개]')
    print(ok.sort_values('PBR차이', ascending=False)[show].head(20).to_string())
    print('\n[PBR 0.2 미만 종목]')
    print(ok[ok['PBR'] < 0.2][show].to_string())
else:
    print(f'\nkor_fs 에 "{P_EQ}" 계정이 없어 지배주주 기준 재계산은 건너뜁니다. 위 지배/비지배 목록에서 이름을 확인하세요.')
```